

Úvod do jazyka Python

Základy

Vladimír Župka 2021

Programovací prostředí.....	6
Proměnné a typy dat	8
Výrazy a příkazy.....	9
Výraz.....	9
Přiřazovací příkaz	9
Rozšířený přiřazovací příkaz	9
Výraz jako příkaz.....	10
Příkaz volání funkce	10
Rozhodovací příkazy	11
Příkaz if.....	11
Příkaz else	12
Vnořené příkazy if a else	12
Příkaz elif.....	12
Prázdný příkaz pass	12
Seznam (list)	13
Čtení podle indexu	13
Porovnání seznamů	13
Výběr podseznamu ze seznamu – slice.....	14
Změna objektů v seznamu	15
Přepis hodnoty na místě indexu	15
Vložení složeného prvku dovnitř seznamu	15
Vložení jednoduchého prvku dovnitř seznamu	15
Odstranění prvku seznamu podle indexu nebo vykrojení – příkaz (statement) del.....	15
Odstranění prvku seznamu podle obsahu – metoda remove()	15
Připojení prvku na konec seznamu – metoda append()	16
Spojení prvků seznamů, operátor + nebo metoda extend()	16
Seřazení seznamu – metoda sort().....	16
Opakovací příkazy	17
Příkaz while	17
Tisk všech položek seznamu.....	17
Tisk čísel pozpátku (odpočítávání)	17
Cyklus se stále splněnou podmínkou.....	17
Příkaz for-in	18
Tisk všech prvků seznamu bez indexu.....	18
Tisk všech prvků seznamu pomocí indexu	18
Tisk čísel 0 1 2 3 4	18
Tisk čísel pozpátku (odpočítávání)	18
Funkce range()	19
Úvodní příklad – maximum ze seznamu čísel.....	20
Zadání	20
Stanovení postupu.....	20
Realizace.....	22
Zjednodušení.....	23
Analýza dat	24
Záporná čísla na vstupu	24
Nečíselné údaje na vstupu.....	24
Cvičení	24
N-tice (tuple).....	25
Čtení prvků (index, slice).....	25

Spojování prvků do nové n-tice operátorem + nebo *	25
Spojování n-tic čárkou	25
Změna měnitelného prvku v n-tici	25
Zabalení a rozbalení n-tice – packing, unpacking	26
Zabalení n-tice - packing	26
Rozbalení n-tice - unpacking	26
Zabalení a rozbalení v jednom přiřazovacím příkazu	26
Rozbalení n-tice z hvězdičkového výrazu v argumentu funkce	26
Převod n-tice na seznam funkcí list()	26
Převod seznamu na n-tici funkcí tuple()	26
Jednočlenná n-tice	26
Množina (set)	27
Operace mezi množinami	27
Operace mezi množinou a posloupnostmi	27
Ostatní operace	27
Funkce – přehled	28
Příklad funkce – kořeny kvadratické rovnice	29
Definice funkce koreny()	29
Volání funkce koreny()	29
Definice funkce	30
Volání funkce	30
Parametry a argumenty	30
Poziční a klíčové parametry	30
Předvolené parametry (default parameters)	31
Vícenásobné parametry (variadic parameters)	31
Texty (znakové řetězce)	32
Řetězcové literály	32
Prázdný řetězec	32
Spojování řetězců do nového řetězce	32
Čtení znaků a vybírání podřetězců	33
Zvláštní znaky se zpětným lomítkem	33
Hrubý řetězcový literál	34
Funkce repr()	34
Vyhledávání v řetězci	34
Zjištění přítomnosti podřetězce	34
Hledání indexů všech výskytů jediného znaku	34
Hledání indexů podřetězce znak po znaku	35
Hledání indexů podřetězce metodou find()	36
Cvičení	36
Porovnání řetězců	37
Proměna řetězce na seznam, n-tici nebo množinu	37
Více o rozhodování	38
Podmínka jako logický výraz	38
Relační operátory	38
Logické operátory	38
Logické výrazy	39
Složitější logické výrazy	40
Aritmeticko-logický výraz	40
Podmínkový výraz	40
Slovník (dictionary)	41

Čtení hodnoty podle klíče	41
Změna hodnoty podle klíče.....	41
Přidání nové položky na konec	41
Odebrání položky podle klíče – funkce del().....	42
Zjišťování informací o klíčích, hodnotách a položkách	42
Převod složek slovníku do seznamu	43
Cvičení	43
Číselné soustavy	44
Převod celého čísla z dvojkové do desítkové soustavy	44
Převod celého čísla z dvojkové do šestnáctkové soustavy	45
Zjednodušený převod mezi dvojkovou a šestnáctkovou soustavou	46
Příklad převodu z desítkové do šestnáctkové soustavy	46
Příklad převodu z desítkové do dvojkové soustavy	47
Cvičení	47
Převodní funkce bin() a hex()	48
Úprava dat pro výstup.....	49
Standardní funkce pro úpravu výstupu	49
Funkce print()	49
Převod objektu na řetězec	49
Převody znaků a pořadových čísel Unicode	50
Formátované řetězcové literály	51
Tvar formátovaného literálu	51
Tvar formátové specifikace	52
Typ zobrazení	52
Příklady formátování	53
Výrazy slice a volání funkce v náhradových polích	53
N-tice a podmínkový výraz ve výrazu	53
Otisk výrazu s výsledkem	53
Formátové specifikace	53
Cvičení	54
Soubory a adresáře.....	55
Cesta (path)	56
Absolutní a relativní cesta.....	56
Běžný adresář (current directory)	57
Vytvoření a smazání adresáře	57
Vytvoření a otevření souboru	58
Zápis dat do souboru funkcí print()	59
Vytvoření souboru a zápis textu metodou write()	59
Test existence a výmaz souboru.....	59
Čtení textového souboru po řádcích	60
Čtení textového souboru po jednom znaku.....	60
Cvičení	60
Chyby a výjimky	61
Zpracování souboru příkazem with-as.....	61
Testování typu dat.....	61
Testování chyb příkazem try	62
Příklad 1	62
Příklad 2	63
Příklad 3	63
Příklad 4	63

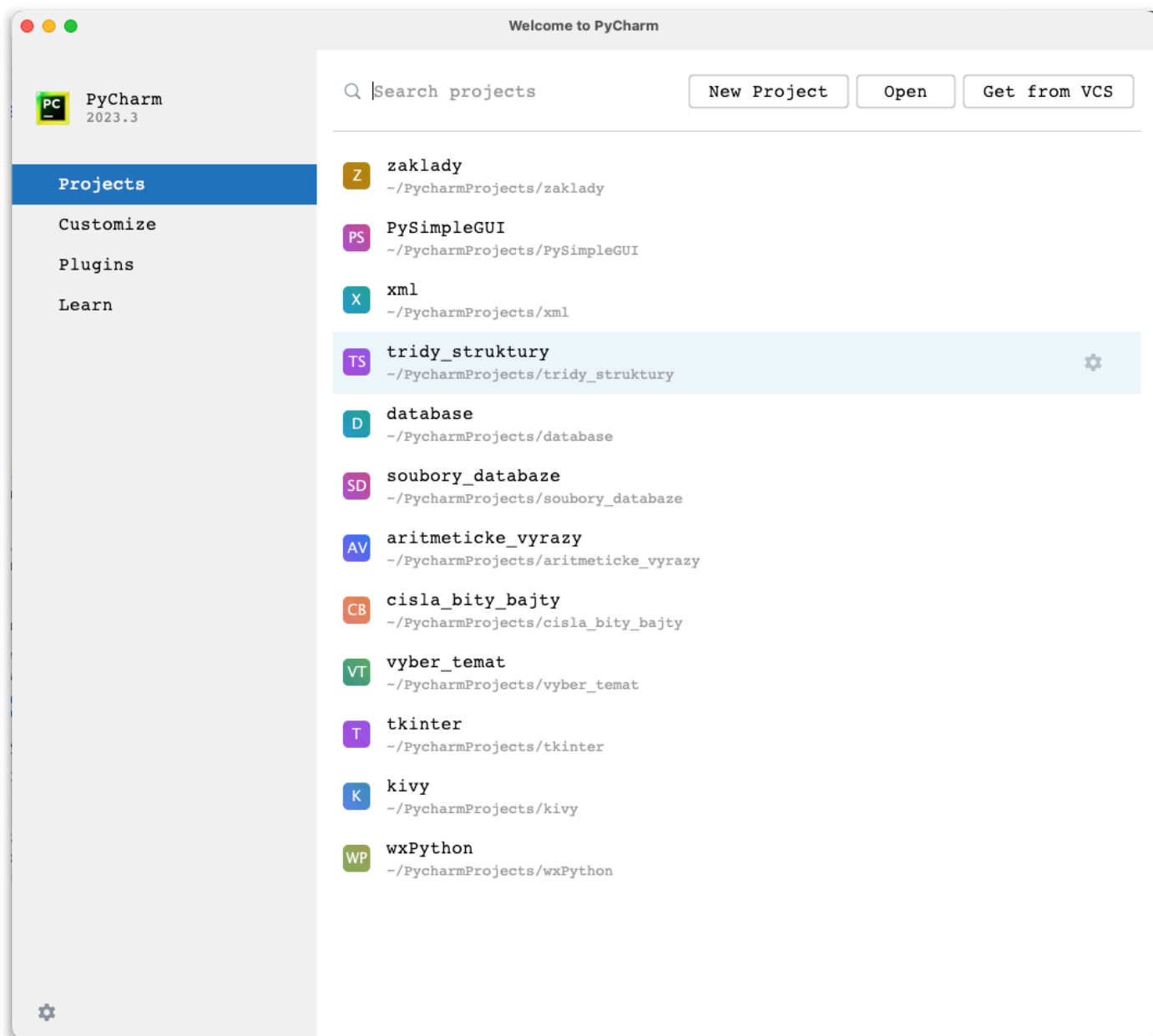
Vyvolání výjimky.....	64
Příkaz assert	64
Příkaz raise.....	65
Vstup dat z klávesnice – funkce input()	66
Příklad zadaný z editoru	66
Příklad zadaný z příkazového řádku	66
Kontrola vstupu (zelená linka uvádí obraz konzoly)	67
Výpočet maxima z čísel zadaných z klávesnice	68

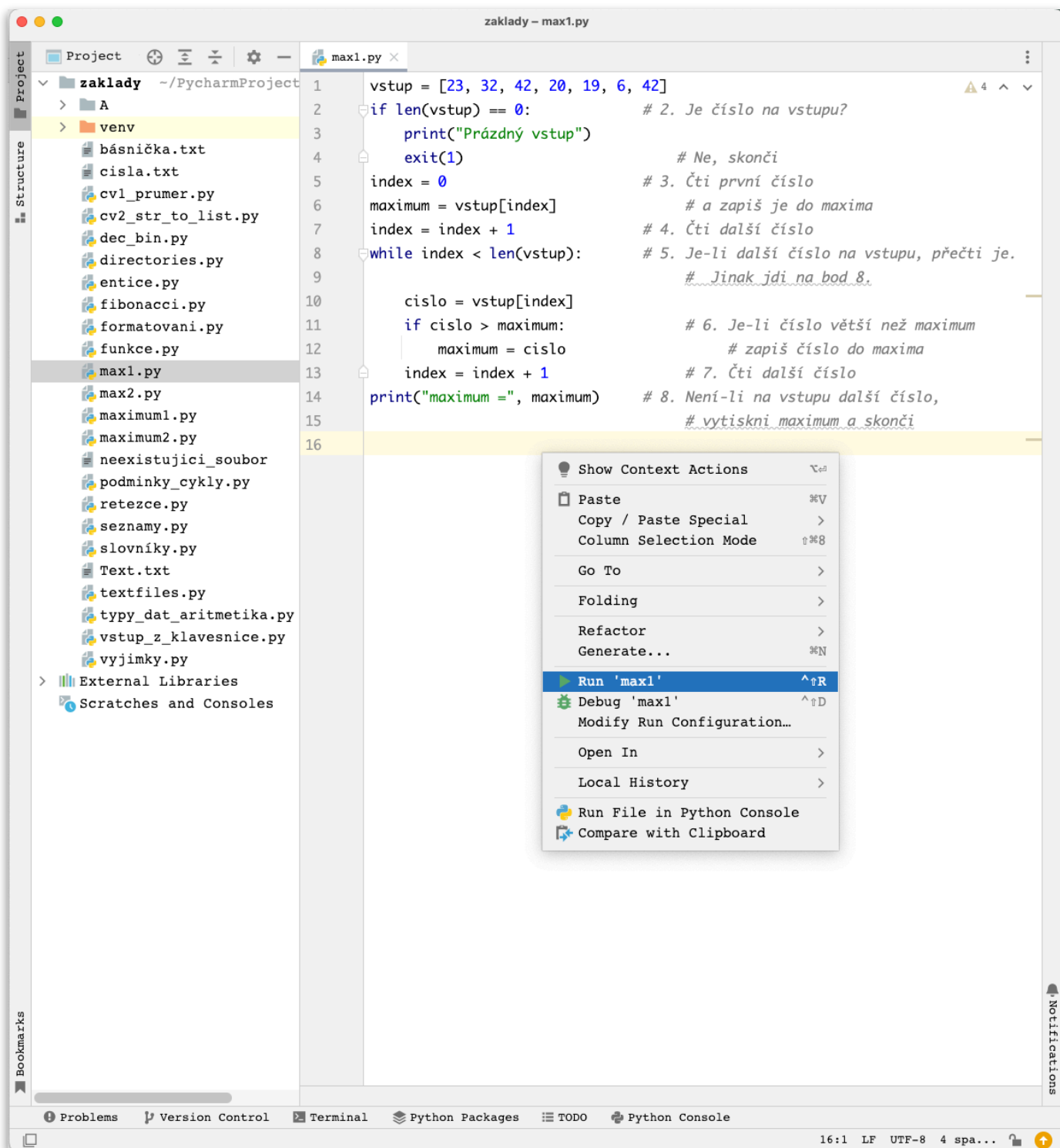
Programovací prostředí

Ze stránky python.org stáhneme poslední verzi jazyka Python pro příslušný operační systém a jeho verzi. Materiály ke školení byly vytvářeny s verzí Python 3.9. Součástí programového balíku je dokumentace a také interakční prostředí IDLE (Integrated Development and Learning Environment).

K vytváření a kontrole příkladů a ukázek používáme vývojové prostředí IDE (Integrated Development Environment). Za nejlepší jsme zvolili projekt *PyCharm* firmy JetBrains. Lze je stáhnout ze stránky

<https://www.jetbrains.com/pycharm/>. Verze *Community* je zdarma.





Proměnné a typy dat

Python pracuje s *daty* v počítači ve formě *objektů*, na něž odkazují *proměnné*. Každý objekt při vytvoření dostává *hodnotu* a *typ*. Nejběžnější typy dat v Pythonu jsou

int integer – celé číslo
float floating point number – zlomkové číslo (číslo s pohyblivou řádovou čárkou)
str string – znakový řetězec (text)

Proměnné mají jména, která se píšou většinou malými písmeny, popřípadě s podtržítky. Povolena je i diakritika. Proměnné získávají přiřazením hodnoty *odkaz* na objekt a jeho typ.

```
tisíc = 1_100
```

Proměnná `tisíc` získává odkaz na objekt s hodnotou `1000` typu `int`.

```
pí = 3.14159
```

Proměnná `pí` získává odkaz na objekt s hodnotou `3.14159` typu `float`.

```
pozdrav = "ahoj"
```

Proměnná `pozdrav` získává odkaz na objekt s hodnotou `ahoj` typu `str`.

Hodnoty `1_000`, `3.14159`, `"ahoj"` se nazývají literály. *Literálem* (litterate – doslova) se nazývá hodnota přímo zapsaná v programu.

```
a = 3.14159      # teď a odkazuje na objekt typu float
```

```
a = "ahoj"      # teď a odkazuje na objekt typu str
```

Stejná proměnná může (v různých okamžicích) odkazovat na objekty různých typů.

Čísla s pohyblivou čárkou mohou mít také tzv. *vědecký formát*. Ten se používá pro velmi malá nebo velmi velká čísla. Když hodnoty nejsou příliš veliké, zobrazí se číslo v obyčejné podobě bez exponentu. Následující příklady jsou z *interakčního režimu* aplikace IDLE.

```
>>> 4.51e-4      # = 4.51 × 10-4
```

```
0.000451
```

```
>>> 5.31e9       # = 5.31 × 109
```

```
5310000000.0
```

```
>>>
```

Číslo `4.51` v příkladu se nazývá *mantisa*. Výraz `e-4` se jmenuje *exponent*, vyjadřuje mocninu deseti a čte se *krát* 10^{-4} . Kladné mocniny lze psát bez znaménka `e9` nebo se znaménkem `e+9`. Čísla s hodně velkými exponenty se již nedají převést na tvar bez exponentu.

Poznámka: Python má tzv. *dynamické typování*, to znamená, že typ proměnné je určen v době výpočtu dosazením hodnoty. Proto jména (proměnné) mohou svůj typ měnit. Jiné jazyky mají většinou tzv. *statické typování*, to znamená, že již při kompilaci stanoví typ proměnné (např. `int`, `char` apod. v jazyku Java), který je stálý během výpočtu.

Výrazy a příkazy

Výraz

Výraz se skládá z operandů a operátorů. *Operandy* jsou objekty v podobě literálů a proměnných, nebo v podobě jiných výrazů. Mohou být uzavřené v závorkách. *Operátory* jsou znaménka pro sčítání, odčítání apod.

```
((2 + 3) * 4)      # 20
```

Výsledek aritmetického výrazu je číslo typu `int`, jsou-li operandy typu `int` a není-li ve výrazu žádné dělení.

```
((2 + 3) * 4) / 2  # 10.0
10 / 1.5           # 6.666666666666667
10 // 1.5          # 6.0 – od podílu odřízne zlomkovou část
10.5 % 3           # 1.5 – zbytek po dělení
10.0 ** 3          # 1000.0 – třetí mocnina
```

Výsledek aritmetického výrazu je číslo typu `float`, jestliže se v něm vykytuje alespoň jeden operand typu `float` nebo operace *dělení*.

Přiřazovací příkaz

Přiřazovací příkaz se skládá z proměnné na levé straně, rovnítka a výrazu na pravé straně.

```
a = 3              # jednoduchý
x, y, z = 2, 3, 4  # násobný (přidělení hodnot proměnným)
```

Rozšířený přiřazovací příkaz

Rozšířený přiřazovací příkaz je přiřazovací příkaz doplněný o aritmetické operátory.

```
x += y    # 5
x -= y    # 2
x *= y    # 6
x /= y    # 2.0
atd.
```

Poznámka: `x += y` znamená totéž jako `x = x + y`.

Výraz jako příkaz

Výraz je také *příkaz*, např. v interakčním režimu.

```
>>> ((2 + 3) * 4)
20
>>>
```

Příkaz volání funkce

Příkaz volání funkce – jméno funkce s argumenty v kulatých závorkách.

```
print( ((2 + 3) * 4) ) # s jedním argumentem

20
```

Funkci `print()` používáme ke kontrolnímu výpisu výsledků. Můžeme ji použít například s argumenty oddělenými čárkou. Toto volání zobrazí na standardním výstupu (konzole) oba vyhodnocené argumenty vedle sebe oddělené mezerou.

```
print("výsledek", ((2 + 3) * 4)) # s dvěma argumenty

výsledek 20

x, y, z = 2, 3, 4 # násobný přiřazovací příkaz
print("((x + y) * z) / 2 =", ((x + y) * z) / 2)

((x + y) * z) / 2 = 10.0
```

Rozhodovací příkazy

Jmenují se také podmínkové nebo podmiňovací příkazy (conditional statements). Jsou složeny z podmínky a bloku příkazů, které se provedou, je-li podmínka splněna. Podmínka je otázka na nějaké tvrzení a dostává hodnotu podle toho, je-li tvrzení pravdivé (hodnota `True`) nebo nepravdivé (hodnota `False`). Tvrzení v podmínce se nazývají *logické výrazy*. Běžné výrazy jsou ty, které porovnávají velikost objektů. Objekty mohou být čísla, texty nebo jiné objekty, které dovolují porovnávat velikost.

<code>a == b</code>	<code>a</code> se rovná <code>b</code> (dvě rovnítka)
<code>a != b</code>	<code>a</code> se nerovná <code>b</code>
<code>a < b</code>	<code>a</code> je menší než <code>b</code>
<code>a > b</code>	<code>a</code> je větší než <code>b</code>
<code>a <= b</code>	<code>a</code> je menší nebo rovné <code>b</code>
<code>a >= b</code>	<code>a</code> je větší nebo rovné <code>b</code>

Příkaz `if`

Příkaz `if` se skládá z podmínky ukončené dvojtečkou a bloku příkazů.

`if` podmínka:
blok příkazů

`a, b = 1, 2`

```
if a < b:                # podmínka - dvojtečka uvádí začátek bloku
    print(a, "<", b)      # blok příkazů musí být odsazen aspoň o 1 pozici
    print(a < b)
```

```
1 < 2
```

```
True
```

Dvojtečka a odsazení (indentation) bloku nahrazuje složené závorky používané v jiných jazycích. Běžné odsazení je o 4 pozice.

V bloku příkazu `if` může být i jiný (vnořený - nested) příkaz `if`.

```
if a <= b:
    if a == b:          # vnořený příkaz
        a = b + 1
    if a < b:            # vnořený příkaz
        a = b
```

```
print(a, b)
```

```
2 2
```

Příkaz **else**

Příkaz **else** se používá ve spojení s příkazem **if**.

```
a, b = 1, 2
```

```
if a >= b:           # jestliže a >= b
    max = a         # dosad' a do max
else:               # jinak
    max = b         # dosad' b do max
```

Vnořené příkazy **if** a **else**

```
if a < b:
    print("min =", a)
else:
    if a == b:       # vnořený příkaz
        print("min =", a)
    else:           # vnořený příkaz
        print("min =", b)
```

```
min = 1
```

Příkaz **elif**

```
if a < b:
    print("min =", a)
elif a == b:        # elif nahradí else s vnořeným if
    print("min =", a)
else:
    print("min =", b)
```

```
min = 1
```

Prázdný příkaz **pass**

Prázdný příkaz **pass** se používá např. tehdy, když potřebujeme vyhradit prázdné místo pro budoucí příkazy nebo vyjádřit žádnou činnost.

```
a, b = 1, 2
c = 0
```

```
if a >= b:          # jestliže a >= b
    c = a           # dosad' a do c
else:              # jinak
    pass           # nedělej nic, c zůstává nezměněno
```

Seznam (list)

Seznam, anglicky *list*, je uspořádaná posloupnost objektů (prvků) libovolných typů. Literál seznamu obsahuje prvky oddělené čárkou a uzavřené v *hrnatých závorkách*. Čárka za posledním objektem se připouští a ignoruje se.

```
s = ["A", "B", "C", "D", "E", ]  
print(s)  # ['A', 'B', 'C', 'D', 'E']
```

Prázdný seznam se zapisuje jako literál `[]` nebo volání funkce `list()` s prázdným argumentem.

Počet prvků v seznamu určí funkce `len()`. (O funkcích jedná pozdější kapitola.)

```
len(s)      # 5 - počet prvků
```

Čtení podle indexu

Prvky seznamu můžeme získat (číst) pomocí pořadových čísel – *indexů* – uzavřených v hranatých závorkách. Indexy začínají *nulou* pro první objekt a končí *počtem objektů minus 1* pro poslední objekt.

```
s[0]        # A - první prvek  
s[1]        # B  
s[4]        # E  
s[5]        # chyba IndexError: list index out of range
```

Indexy mohou být i záporné. Záporné indexy (od konce) začínají -1 a končí *záporným počtem objektů*.

```
s[-1]       # E - poslední prvek  
s[-2]       # D  
s[-5]       # A  
s[-6]       # chyba IndexError: list index out of range
```

Porovnání seznamů

Seznamy se porovnávají podle velikosti stejnohlých prvků, jestliže se ovšem dají porovnávat. V příkladu je seznam `s` větší než seznam `t`, protože prvek `s[1]` je větší než `t[1]` a na dalších už nezáleží.

```
s = [1, 2, 2, 2, ]  
t = [1, 0, 3, 4, 5]  
  
print(s > t)    # logický výraz "seznam s je větší než seznam t"  
  
True
```

Výběr podseznamu ze seznamu – `slice`

Výběr podseznamu (`slice` – vykrojení, výkrojek, výřez) se děje pomocí následujících výrazů.

```
[start : stop]
[start : stop : step]
```

Údaj `start` je *index* prvního vykrajovaného prvku,

údaj `stop` je *index* posledního vykrajovaného prvku *zvětšený o 1*,

údaj `step` je *krok* (číslo, o které se zvyšuje index vykrajovaného prvku).

Každý z údajů lze vynechat, mají předem zvolené hodnoty:

`start` = 0, `stop` = počet prvků seznamu, `step` = 1.

```
s = ["A", "B", "C", "D", "E"]
```

A	B	C	D	E
0	1	2	3	4
-5	-4	-3	-2	-1

```
print(s[:2])          # ['A', 'B']
print(s[2:])          # ['C', 'D', 'E']
print(s[1:3])         # ['B', 'C']
print(s[1::2])        # ['B', 'D']
print(s[1:len(s):2])  # ['B', 'D']
print(s[-4:-1])       # ['B', 'C', 'D']
print(s[:])           # ['A', 'B', 'C', 'D', 'E']
print(s[0:5])         # ['A', 'B', 'C', 'D', 'E']
```

Některé zvláštní případy indexů a výřezů.

Prvky:

`s[0]` získá *první* prvek seznamu 'A'

`s[-1]` získá *poslední* prvek 'E'

Podseznamy:

`s[0:1]` získá podseznam s *prvním* prvkem ['A'] stejně jako `s[0]`,

`s[-1:]` získá podseznam s *posledním* prvkem ['E'] stejně jako `s[-1]`

`s[0:0]` získá *prázdný* podseznam [],

`s[:0]` získá *prázdný* podseznam [],

`s[0:]` získá celý seznam stejně jako `s[:]`, `s[::]`, nebo jako `s` bez indexu,

`s[:-1]` získá podseznam od začátku do *předposledního* objektu ['A', 'B', 'C', 'D'],

Změna objektů v seznamu

Přepis hodnoty na místě indexu

```
s = ["A", "B", "C", "D", "E"]
s[1] = 56.1 # ['A', 56.1, 'C', 'D', 'E']
```

Vložení složeného prvku dovnitř seznamu

Vkládaný prvek musí být schopný iterace (postupného přístupu). Může to být například podřetězec nebo text, který se vkládá do pozice za výřezem tvaru `[i:i]` a je přitom rozčleněn na své jednotlivé složky.

```
s = ["A", "B", "C", "D", "E"]
s[2:2] = [10, 20] # ['A', 'B', 10, 20, 'C', 'D', 'E']
s[2:2] = 'ab' # ['A', 'B', 'a', 'b', 10, 20, 'C', 'D', 'E']
```

Vložení jednoduchého prvku dovnitř seznamu

Jednoduchý prvek se ukládá za zadaný výřez (slice)

```
s = ["A", "B", "C", "D", "E"]
s[2:2] = 'b' # ['A', 'B', 'b', 'C', 'D', 'E']
```

Vkládaný prvek musí být *schopný iterace* (postupného přístupu)! Číslo není, zatímco text nebo seznam je. V následujícím příkladu tedy vkládané číslo dáváme do hranatých závorek:

```
s[0:0] = [100] # [100, 'A', 'B', 'b', 'C', 'D', 'E']
```

Prvek můžeme vložit také *metodou* **insert()**. Metoda `s.insert(i, x)` dává podobný výsledek jako vykrojení `s[i:i]`. Má-li ale vložený prvek více složek nerozčleňuje se na složky.

```
s.insert(1, 'cd') # [100, 'A', 'cd', 'B', 'b', 'C', 'D', 'E']
s.insert(0, [3, 4]) # [[3, 4], 100, 'A', 'cd', 'B', 'b', 'C', 'D', 'E']
```

Poznámka: Seznam (jako další typy objektů) je objekt definovaný jako třída (class). *Metoda* je funkce definovaná uvnitř třídy. Volání metody je tedy kvalifikováno (uvedeno) jménem objektu s tečkou. O třídách bude pojednáno v dalších částech kurzu.

Odstranění prvku seznamu podle indexu nebo vykrojení – příkaz (statement) **del**

Poznámka: Příkaz `del` není ani funkce ani metoda (a také nemá závorky pro argumenty).

```
s = ["A", "B", "C", "D", "E"]
del s[0] # ["B", "C", "D", "E"]
del s[0:1] # ["C", "D", "E"]
```

Odstranění prvku seznamu podle obsahu – metoda **remove()**

```
s = ["A", "B", "C", "D", "E"]
s.remove("C") # ["A", "B", "D", "E"]
```

Připojení *prvku* na konec seznamu – metoda `append()`

```
s = ["A", "B", "C", "D", "E"]
s.append("F")
s.append("G")
print(s)  # ['A', 'B', 'C', 'D', 'E', 'F', 'G']

pridane = list()  # prázdný seznam, stejné jako []
pridane.append(["jablka", 56.1])
pridane.append(["hrušky", 67.0])
pridane.append(["švestky", 30.99])
pridane.append(["broskve", 85.00])

print(pridane)
[['jablka', 56.1], ['hrušky', 67.0], ['švestky', 30.99], ['broskve', 85.0]]
```

Prvky byly seznamy. Vznikl *seznam seznamů* (dvojměrný seznam).

```
pridane[0][0]  # jablka
pridane[3][1]  # 85.0
```

Spojení prvků seznamů, operátor `+` nebo metoda `extend()`

```
spojene_prvky = ["jablka", 56.1] + ["hrušky", 67.00]
['jablka', 56.1, 'hrušky', 67.0]

spojene_prvky += ["švestky", 30.99]
['jablka', 56.1, 'hrušky', 67.0, 'švestky', 30.99]

spojene_prvky.extend(["broskve", 85.00])  # stejné jako +=
['jablka', 56.1, 'hrušky', 67.0, 'švestky', 30.99, 'broskve', 85.0]
```

Setřídění seznamu – metoda `sort()`

```
s = ["c", "bb", "aaa"]
s.sort()  # podle velikosti prvků lexikograficky
print(s)  # ['aaa', 'bb', 'c']

s.sort(key=len)  # podle délky prvků
print(s)  # ['c', 'bb', 'aaa']
```


Opakovací příkazy

Jmenují se také příkazy cyklu, smyčky (cycle, loop). Za určité podmínky opakují skupinu (blok) příkazů.

Příkaz `while`

Příkaz `while` (česky zatímco) má tvar

```
while podmínka:
    blok příkazů
```

Obsahuje podmínku (podobně jako příkaz `if`) ukončenou dvojtečkou a odsazený blok příkazů. Dokud je podmínka splněná (`True`), příkazy bloku se provádějí. Jakmile přestane podmínka platit (`False`), cyklus skončí.

Tisk všech položek seznamu

```
s = ["A", "B", "C", "D", "E"]
i = 0
while i < len(s):    # tisknou se všechny položky seznamu s
    print(s[i], end=", ") # po čísle a se tiskne čárka a mezera ve stejném řádku
    i += 1
A, B, C, D, E,
```

Tisk čísel pozpátku (odpočítávání)

```
a = 10:
while a > 0:    # tisknou se čísla od 10 do 1
    print(a, end=", ")
    a -= 1
10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
```

Cyklus se stále splněnou podmínkou

"Nekonečný" cyklus s příkazy `continue` (pokračuj) a `break` (přeruš). Podmínka u `while` je stále pravdivá – `True`.

```
a = 10
while True:    # tisknou se čísla od 10 do 1
    if a > 0:
        print(a, end=", ")
        a -= 1    # odečtení jedničky
        continue    # skok zpět na začátek cyklu
    else:
        break    # skok dopředu ven z cyklu
10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
```

Příkaz **for-in**

Příkaz má tvar

```
for prvek in objekt:  
    blok příkazů
```

Prochází postupně všechny *prvky* v *objektu*. Objektem může být každý objekt složený z více prvků umožňující získávat tyto prvky postupně (iterovat). Takovým objektem je např. *seznam* nebo funkce *range* (rozsah, řada).

Tisk všech prvků seznamu bez indexu

```
s = ["A", "B", "C", "D", "E"]  
for element in s:  
    print(element, end="")
```

A B C D E

Tisk všech prvků seznamu pomocí indexu

```
for n in range(len(s)):  
    print(s[n], end=" ")
```

A B C D E

Tisk čísel 0 1 2 3 4

```
for n in range(5):  
    print(n, end=" ")
```

0 1 2 3 4

Tisk čísel pozpátku (odpočítávání)

```
for n in range(5, 0, -1):  
    print(n, end=" ")
```

5 4 3 2 1

```
start = 4; stop = -5; step = -2  
for n in range(start, stop, step):  
    print(n, end=" ")
```

4 2 0 -2 -4

Funkce `range()`

Funkce má tři tvary: `range(stop)`
`range(start, stop)`
`range(start, stop, step)`

Funkce vrací posloupnost celých čísel začínajících hodnotou *start*, pokračujících po přírůstku *step* a končících ještě před tím než nabudou hodnoty *stop*. Není-li zadán argument *start*, dosadí se za něj 0. Není-li zadán argument *step*, dosadí se za něj 1.

Příklady pořadových čísel z funkce `range()`.

<code>range(5)</code>	0 1 2 3 4
<code>range(0, -5, -1)</code>	0 -1 -2 -3 -4
<code>range(-4, 1)</code>	-4 -3 -2 -1 0
<code>range(-4, 0)</code>	-4 -3 -2 -1
<code>range(-3, 2)</code>	-3 -2 -1 0 1
<code>range(1, 5, 2)</code>	1 3

Úvodní příklad – maximum ze seznamu čísel

Zadání

Máme úkol sestavit program pro výběr největšího čísla ze zadaného seznamu čísel.

Stanovení postupu

Je-li seznam dost krátký, okamžitě vidíme největší číslo. Když ale bude seznam velký, nepřehlédneme jej náraz, a proto musíme postupovat systematicky. Tak například:

Probíráme čísla od začátku seznamu a zapamatujeme si první číslo v pořadí, za nímž následuje menší číslo. Podíváme se dále a probíráme další čísla a vynecháme všechna čísla, která jsou stejně velká nebo menší než to zapamatované číslo. Když najdeme větší číslo, zapamatujeme si je. Zase se podíváme dále atd. až do vyčerpání seznamu.

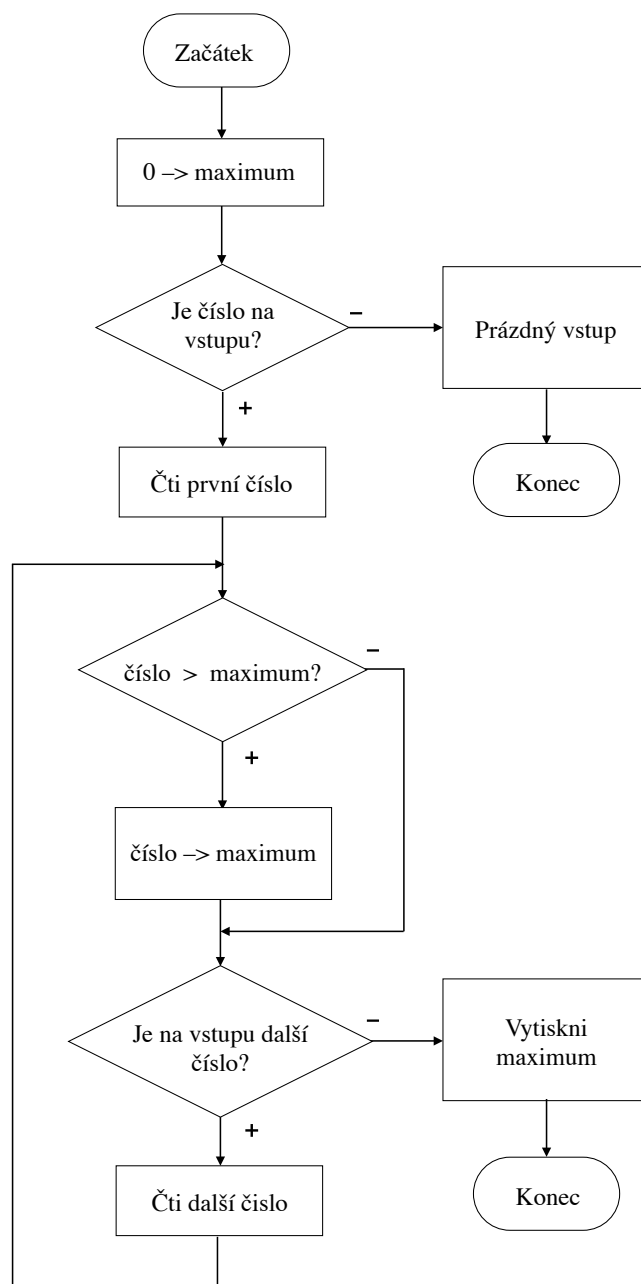
Pro počítač musíme návod upravit, protože počítač musí čísla odněkud číst a jedno po druhém porovnávat. Představme si, že čísla jsou připravena ve frontě na vstupu do zpracování. Čtení čísla bude znamenat jeho vyjmutí ze vstupu a jeho nahrazení dalším číslem z fronty.

1. Urči místo pro uložení maxima.
2. Přečti číslo, které je na vstupu, ulož je do maxima.
3. Přečti další číslo ze vstupu a porovnej je s tím maximem.
4. Je-li nové číslo větší, přepiš jím dosavadní maximum a opakuj od bodu 3.

Vidíme, že se některé kroky opakují, ale z návodu není zřejmé, kdy má opakování skončit, jestli vůbec. Kromě toho nepočítáme s tím, že vstup může být prázdný. Zkusíme návod zpřesnit.

1. Urči místo pro uložení maxima a dosad' do něj nulu.
2. Není-li na vstupu žádné číslo, ohlas to tiskem a skonči.
3. Přečti první číslo ze vstupu.
4. Je-li číslo větší než maximum, zapiš číslo do maxima.
5. Je-li na vstupu další číslo, přečti je a opakuj od bodu 4.
6. Není-li již na vstupu další číslo, vytiskni maximum a skonči.

Na další stránce je vyjádření tohoto předpisu vývojovým diagramem (flow-chart). Takové diagramy bývaly povinnou složkou dokumentace, existovaly na jejich rýsování šablony a pro sjednocení dokonce státní norma. Vznikly v době, kdy se programovalo ve strojových instrukcích, kdy byl pro návrh a psaní programu k dispozici jen papír a tužka (případně programové formuláře).



Realizace

Podíváme se na to, jak jsou čísla zadána, jakým způsobem se bude číst první a další číslo ze vstupu a jak se bude zkoumat, zda existuje další číslo na vstupu. To už je věc programovacího jazyka.

Pro začátek budeme mít seznam čísel [23, 32, 42, 20, 19, 6, 42] zapsaný v programu. Každé číslo v seznamu má svoje pořadové číslo zvané *index*, který začíná nulou a končí jako počet čísel minus 1. Čtení dalšího čísla se tedy děje tak, že se index zvýší o 1 a ze vstupu se vybere číslo na pozici indexu.

Náš předpis vyžaduje, aby se výpočet přecházel z jednoho místa zpět na jiné místo, což se v počítači uskuteční příkazem *skoku* na místo označené *návěstím* (*label*). Takový příkaz je k dispozici ve strojovém jazyku a v některých vyšších jazycích, např. v jazyku C. V nejnovějších jazycích včetně jazyka Python však chybí.

Na ukázkou skokového příkazu je zde program napsaný v jazyku C bez opakovacího příkazu, ale s využitím skokového příkazu `goto`.

```
#include <stdio.h>

int main(int argc, const char * argv[]) {
    int vstup[] = {23, 32, 42, 20, 19, 6, 42}; // seznam čísel
    int maximum = 0;                          // 1. Urči místo pro uložení maxima.
    int cislo = 0;
    int pocet = sizeof(vstup)/sizeof(int); // počet čísel v seznamu
    if (pocet == 0) {                          // 2. Je číslo na vstupu?
        printf("Prázdný vstup.\n");
        goto konec;                          // Ne, jdi na konec.
    }
    int index = 0;                             // 3. Čti první číslo.
    cislo = vstup[0];
opakovat:
    if (cislo > maximum) {                      // 4. Je číslo větší než maximum?
        maximum = cislo;                      // Ano, zapiš číslo do maxima.
    }
    index = index + 1;
    if (index < pocet) {                      // 5. Je na vstupu další číslo?
        cislo = vstup[index];                // Ano, čti další číslo.
        goto opakovat;                      // a vrať se k bodu 4.
    }
    printf("maximum = %d\n", maximum); // 6. Vytiskni maximum a skonči.
konec:
    return 0;
}
```

V jazyku Python můžeme k opakování příkazů použít příkaz **while** nebo **for-in**. Budeme-li se držet postupu podle vývojového diagramu, použijeme příkaz **while**. Použijeme také příkazy **continue** a **break**, což jsou v podstatě také skokové příkazy, ale bez návěstí a omezené na použití v bloku.

```
vstup = [23, 32, 42, 20, 19, 6, 42]
maximum = 0                                # 1. Urči místo pro uložení maxima.
if len(vstup) == 0:                         # 2. Je číslo na vstupu?
    print("Prázdný vstup.")
    exit(1)                                # Ne, skonči.
index = 0                                  # 3. Čti první číslo.
cislo = vstup[0]
while True:
    if cislo > maximum:                     # 4. Je číslo větší než maximum?
        maximum = cislo                    # Ano, zapiš číslo do maxima.
    index = index + 1                       # 5. Je na vstupu další číslo?
    if index < len(vstup):
        cislo = vstup[index]               # Ano, přečti číslo a vrať se k bodu 4.
        continue
    else:
        print("maximum =", maximum)        # 6. Ne, vytiskni maximum a skonči.
        break

maximum = 42
```

Zjednodušení

Řešení úlohy můžeme velmi zjednodušit tím, že použijeme příkaz **for-in**. Nemusíme pak zjišťovat počet položek seznamu, indexovat položky, zvyšovat index a testovat jej.

```
vstup = [23, 32, 42, 20, 19, 6, 42]
maximum = 0
if len(vstup) == 0:
    print("Prázdný vstup.")
    exit(1)    # opustí program se zvoleným kódem 1
for cislo in vstup:
    if cislo > maximum:
        maximum = cislo
print("maximum =", maximum)

maximum = 42
```

Analýza dat

Záporná čísla na vstupu

Co když se na vstupu vyskytnou samá záporná čísla? Maximum by vyšlo nula, což by nebylo správné. Opravíme program tak, aby vzal v úvahu i tento případ. Pro stručnost si teď odpustíme testování prázdného vstupu a budeme předpokládat, že existuje první číslo. Místo nastavení počáteční hodnoty maxima na nulu přečteme do maxima první číslo.

```
vstup = [-23, -32, -42, -20, -19, -6, -42]
maximum = vstup[0]          # do maxima dosadíme první číslo ze vstupu
for cislo in vstup[1:]:     # stačí od druhého čísla do konce
    if cislo > maximum:
        maximum = cislo
print("maximum =", maximum)

maximum = -6
```

Nečíselné údaje na vstupu

Zatím jsme věřili, že v zadaném vstupu jsou jen čísla. Co se stane, když ve vstupním seznamu budou i nějaké jiné objekty? Zkusme místo některého čísla zadat text, např. "dvacet".

```
vstup = [23, 32, 42, "dvacet", 19, 6, 42]
maximum = vstup[0]
for cislo in vstup[1:]:
    if cislo > maximum:
        maximum = cislo
print("maximum =", maximum)
```

Po spuštění programu Python ohlásí chybu, že nelze porovnávat řetězec s číslem:

```
Traceback (most recent call last):
  File "/Users/vzupka/PycharmProjects/zaklady/maximum2.py", line 15, in <module>
    if cislo > maximum:
TypeError: '>' not supported between instances of 'str' and 'int'
```

Python je jeden z jazyků, které mají tzv. *dynamické typování*. Typ proměnné není určen při překladi (kompilaci) programu, ale je určen až *v průběhu výpočtu*, což znamená, že také kontrola typu je možná jen při výpočtu. Chyba typu je zjištěna, až když dojde k pokusu o nepovolenou operaci. Jak se takovým chybám vyhnout, bude vysvětleno [později](#).

Cvičení

Sestavte program pro výpočet *součtu* a *průměru* z čísel zadaných v seznamu.

N-tice (tuple)

N-tice (anglicky *tuple*) je uspořádaná posloupnost objektů (prvků) libovolných typů. Literál *n-tice* se zapisuje jako seznam prvků oddělených čárkou. Čárka za posledním objektem se připouští a ignoruje se. Prvky *mohou*, ale *nemusí* být uzavřeny v *kulatých závorkách*.

N-tice je podobná seznamu až na to, že její *obsah nelze měnit*. Jsou-li ale její prvky měnitelné objekty (například seznamy), tyto prvky lze měnit.

```
ent = (1, 2, 3, 4)
ent = 1, 2, 3, 4
```

Prázdná *n-tice* se zapisuje jako **()** nebo **tuple()**.

```
ent = 10,      # jednočlenná n-tice (s koncovou čárkou)
ent = (10,)    # jednočlenná n-tice (s koncovou čárkou)
```

Bez koncové čárky by zápis znamenal číslo 10.

Čtení prvků (index, slice)

Je stejné jako u seznamů, podle indexu nebo vykrojení (slice).

```
ent = 1, "a", 3, "e", "8.7",

ent[1]          # a
ent[len(ent) - 1] # 8.7 - poslední prvek
ent[1: 3]       # ('a', 3)
```

Spojování prvků do nové *n-tice* operátorem + nebo *

```
ent1 = 1, "a", 3,
ent2 = ("e", "8.7",)

nova_entice = ent1 + ent2 # připojení prvků (1, 'a', 3, 'e', '8.7')
nova_entice = ent1 * 2    # násobení n-tice (1, 'a', 3, 1, 'a', 3)
```

Spojování *n-tic* čárkou

```
print(ent1, ent2)          # ((1, 'a', 3), ('e', '8.7'))
```

Změna měnitelného prvku v *n-tici*

Je-li prvkem *n-tice* měnitelný objekt, např. seznam, lze jej měnit.

```
sez = [1, 2]
ent = (sez, 3) # ([1, 2], 3)

ent = (["a", "b"], 3) # (['a', 'b'], 3)
```

Zabalení a rozbalení n-tice – packing, unpacking

Zabalení n-tice - packing

```
ent = 1, 2, 3
print(ent)  # (1, 2, 3)
```

Rozbalení n-tice - unpacking

Počet rozbalovaných proměnných musí být totožný s počtem prvků n-tice.

```
a, b, c = ent
print(a, b, c)  # 1 2 3
```

Zabalení a rozbalení v jednom přiřazovacím příkazu

```
a, b, c = 1, 2, 3
```

Je to vlastně násobný přiřazovací příkaz.

Rozbalení n-tice z hvězdičkového výrazu v argumentu funkce

```
print(*ent)  # 1 2 3 – '*ent' je hvězdičkový výraz (starred expression)
```

Převod n-tice na seznam funkcí `list()`

```
seznam = list((1, 2, 3))  # převod entice na seznam
print(seznam)  # [1, 2, 3]
```

Převod seznamu na n-tici funkcí `tuple()`

```
entice = tuple(seznam)  # převod seznamu na n-tici
print(entice)  # (1, 2, 3)
```

Jednočlenná n-tice

```
ent = 10,  # jednočlenná n-tice (musí mít koncovou čárku)
print(*ent)  # 10 – hvězdičkový výraz

a = ent[0]  # a = 10 – nultý člen n-tice
a, = ent    # a = 10 – převod jednočlenné n-tice
(a,) = ent  # a = 10 – převod jednočlenné n-tice
(a,) = tuple(ent)  # a = 10 – převod jednočlenné n-tice
[a] = ent     # a = 10 – převod jednočlenné n-tice do seznamu
[a] = list(ent)  # a = 10 – převod jednočlenné n-tice do seznamu
```

Množina (set)

Množina se skládá z objektů, které jsou *neuspořádané* a jsou *jednoznačné*. Nesmí tedy obsahovat dva objekty se stejnou hodnotou. Množinu lze vytvořit jednak zápisem hodnot do *složených závorek* (set literal) nebo použitím standardní funkce `set()` s nějakou posloupností v argumentu (např. seznam, n-tice, textový řetězec).

```
mnozina1 = {"a", "b", "c", "d", "e"} # literál
print(mnozina1) # {'e', 'a', 'b', 'c', 'd'}

seznam = ["a", "b", "c", "d", "e"]
mnozina2 = set(seznam)
print(mnozina2) # {'e', 'c', 'd', 'a', 'b'}

prazdna_mnozina = set() # ne z literálu {}
```

Obvyklé použití množiny je *unifikace* jiných posloupností (např. seznamů, n-tic) použitím standardní funkce `set()`, to znamená, *odstranění duplicitních objektů* a vytvoření množiny ze zbylých objektů.

```
ntice = ("a", "b", "d", "b", "b", "c", "d", "e", "e")
mnozina3 = set(ntice) # unifikace prvků n-tice
print(mnozina3) # {'e', 'c', 'd', 'a', 'b'}
```

Operace mezi množinami

$a \& b$ průnik (intersection)
 $a | b$ sjednocení (union)
 $a - b$ rozdíl (difference)
 $a ^ b$ symetrický rozdíl (symmetric difference)

```
set1 = {1, 2, 3, 3, 4, 4} # {1, 2, 3, 4}
set2 = {5, 5, 2, 4, 6} # {2, 4, 5, 6}

print(set1 & set2) # {2, 4}
print(set1 | set2) # {1, 2, 3, 4, 5, 6}
print(set1 - set2) # {1, 3}
print(set1 ^ set2) # {1, 3, 5, 6}
```

Operace mezi množinou a posloupnostmi

```
print(set1.intersection([5, 5, 2, 4, 6])) # {2, 4}
print(set1.union([5, 5, 2, 4, 6])) # {1, 2, 3, 4, 5, 6}
print(set1.difference([5, 5, 2, 4, 6])) # {1, 3}
print(set1.symmetric_difference([5, 5, 2, 4, 6])) # {1, 3, 5, 6}
```

Ostatní operace

Existuje mnoho dalších operací s množinami, viz [dokumentaci](#). Například metody `add()` pro přidání objektu k množině, apod.

Funkce – přehled

V programovacích jazycích se používá konstrukce, která umožňuje provést několikrát stejný nebo podobný sled příkazů, aniž by se musel v textu opakovat. Nazývá se různými jmény, např. podprogram, procedura, funkce, metoda apod. Její průběh se modifikuje prostřednictvím parametrů a výsledkem může být nějaká hodnota nebo jiný účinek. V Pythonu se pro takovou konstrukci užívá název *funkce*.

Funkce v Pythonu odpovídá matematické definici funkce $y = f(x)$ resp. $y = f(x_1, x_2, \dots)$. Například lineární funkce $y = ax + b$ je v matematice funkcí jedné nezávisle proměnné x .

Pro příklad však za konstanty a, b volíme konkrétní hodnoty, protože s písmeny nelze provádět aritmetické operace, např. $y = 2x + 1$. V programu funkci napíšeme takto:

```
def f(x):  
    y = 2 * x + 1 # blok příkazů - tělo funkce  
    return y
```

Funkce je výpočetní objekt, ale sama se nespouští. Funkci je třeba vyvolat příkazem *volání funkce*:

```
f(2)      # vrátí hodnotu 5
```

Poznámka:

Funkce je samostatný objekt, na rozdíl od *metody*, která je součástí třídy (class). O třídách a metodách se pojednává v další části kurzu. Volání metody je kvalifikováno objektem, tj. jménu metody předchází jméno objektu s tečkou. Například seznam `sez` vzniká voláním třídy `list`:

```
sez = list()
```

Proto *metoda* `append()` se musí volat s předponou `sez.`

```
sez.append("A")
```

```
sez.append("B")
```

```
print(sez)    # ['A', 'B']
```

Příklad funkce – kořeny kvadratické rovnice

Vypočítejte reálné kořeny rovnice $a^2x + bx + c = 0$. Kořeny se vypočtou podle vzorce

$x_1, x_2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$. Je-li diskriminant $b^2 - 4ac$ kladný, má rovnice dva různé kořeny. Je-li nulový, má rovnice jeden kořen. Je-li záporný, nemá žádný kořen (v reálných číslech).

Definice funkce `koreny()`

Funkci nazveme *koreny* a definujeme ji se třemi parametry a, b, c takto:

```
def koreny(a, b, c):
    """Výpočet kořenů kvadratické rovnice
    a*x**2 + b*x + c = 0"""
    if a == 0: return "Není to kvadratická rovnice." # jednořádkový blok
    diskriminant = (b**2 - 4*a*c)
    if diskriminant < 0: return "Nemá řešení v reálných číslech."
    if diskriminant > 0:
        x1 = (-b + diskriminant ** 0.5) / (2 * a)
        x2 = (-b - diskriminant ** 0.5) / (2 * a)
        return x1, x2 # vrací dvojici čísel (dvojčlennou n-tici)
    if diskriminant == 0:
        x = (-b + diskriminant ** 0.5) / 2 * a
        return x # vrací jedno číslo
```

Text uzavřený v trojitých uvozovkách na začátku bloku je *dokumentační řetězec*, který lze vypsát následujícím příkazem.

```
print(koreny.__doc__)
Výpočet kořenů kvadratické rovnice
a*x**2 + b*x + c = 0
```

Volání funkce `koreny()`

```
kor = koreny(-1, 2, -1) # argumenty bez jmen parametrů v přesném pořadí
print(kor) # 1.0

kor = koreny(1, c = 1, b = 2) # první argument bez jména, ostatní se jmény
print(kor) # -1.0

kor = koreny(1, 3, c = 1) # první dva argumenty bez jména, poslední se jménem
print(kor) # (-0.3819660112501051, -2.618033988749895)

kor = koreny(c = 1, b = 1, a = 1,) # všechny argumenty se jménem
print(kor) # Nemá řešení v reálných číslech.
```

Všimněme si, že funkce může vracet různé typy objektů (číslo, dvojice čísel, text).

Definice funkce

```
def jméno-funkce(parametry):  
    blok příkazů
```

Definice funkce začíná symbolem `def`. Za ním je *jméno funkce* a *parametry* uzavřené v závorkách a oddělené čárkou (čárka za posledním parametrem se ignoruje). Následuje *dvojtečka*, která uvádí *blok příkazů* (zcela stejně jako u podmínkových nebo cyklických příkazů). Blok se nazývá také *tělo funkce*. Funkce může vracet hodnotu příkazem `return`.

Funkce nemusí mít žádné parametry (ale závorky jsou povinné), ani nemusí nic vracet (v tom případě Python vrací hodnotu `None`).

Volání funkce

```
jméno-funkce(argumenty)
```

Volání funkce se skládá ze *jména funkce* následovaného *argumenty* v závorkách. Argumenty odpovídají parametrům z definice funkce. Volání funkce ve výrazu je nahrazeno hodnotou vrácenou příkazem `return` nebo hodnotou `None`.

Parametry a argumenty

Poziční a klíčové parametry

V definici funkce se v závorkách zadávají *parametry* svými *jmény*. Ve volání funkce parametrům odpovídají *argumenty*. Mohou být zadány jako hodnota bez jména nebo jako jméno s hodnotou.

Argumenty bez jmen parametrů se nazývají *poziční* (*positional*) a musí přesně odpovídat pořadí parametrů z definice funkce.

Argumenty se jmény parametrů se nazývají *klíčové* (*keyword arguments*) a nemusí odpovídat pořadí z definice funkce.

Všechny poziční argumenty musí předcházet klíčovým. Čárka za posledním argumentem se ignoruje.

Při dodržení těchto pravidel lze poziční a klíčové argumenty libovolně kombinovat.

```
def kombinované(arg1, arg2):  
    print(arg1, arg2)
```

```
kombinované(2, 3)           # oba poziční           – tiskne 2 3  
kombinované(2, arg2=3)      # poziční a klíčový – tiskne 2 3  
kombinované(arg1=2, arg2=3) # oba klíčové         – tiskne 2 3
```

Předvolené parametry (default parameters)

```
def předvolený(arg = 5): # předvolená hodnota
    print(arg)
```

```
předvolený()      # bez argumentu tiskne 5
předvolený(11)    # s argumentem tiskne argument 11
```

Vícenásobné parametry (variadic parameters)

Vícenásobný parametr je označen předřazenou hvězdičkou (starred expression). Může jich být více. Ve volání funkce pak odpovídající argumenty *nesmějí mít jména*.

```
def maximum(nadpis, *cisla): # parametr nadpis je také povinně poziční
    print(nadpis, end=' ')
    max = 0
    for n in cisla: # jméno parametru je vlastně n-tice
        if n > max: max = n # blok je na stejném řádku se záhlavím příkazu if
    return max
```

```
print(maximum("maximum:", 1, 2, 3.0, 4)) # argumenty tvoří n-tici (čtveřici)
maximum: 4
```

Texty (znakové řetězce)

Textové údaje se vyjadřují znakovými řetězci. *Znakový řetězec* (string) je objekt typu `str` a skládá se ze *znaků* (*characters*). Jednotlivé znaky jsou řetězce o jednom znaku. Znaky jsou kódovány v soustavě *Unicode*, speciálně typu *UTF-8*. Proto je možné zpracovávat libovolné znaky světových jazyků, a dokonce obrázkové symboly. Každý znak má určené pořadové číslo. Například

A má pořadové číslo 65
a má pořadové číslo 97
🐱 má pořadové číslo 128049
🐶 má pořadové číslo 128054

Řetězcové literály

```
"abcd"      # uvozovky (dvojitě)
'abcd'      # uvozovky jednoduché (apostrofy)

"a"         # znak a
'a'

" 'ab' 'cd' " # apostrofy uvnitř uvozovek
' "ab" "cd" ' # uvozovky uvnitř apostrofů

""" Víceřádkový text
   je uzavřen
   ve trojitých uvozovkách nebo apostrofech."""
```

Prázdný řetězec

Je vyjádřen literálem `""` nebo `' '`. Prázdný řetězec se používá mj. k označení konce textového souboru.

Poznámka: Existují také *bajtové řetězce*, které se označují předponou `b`, např. `b"a1"`. Prázdný bajtový řetězec má literál `b""` nebo `b' '`. Ten se používá k označení konce *binárního souboru*.

Spojování řetězců do nového řetězce

Znakový řetězec je neměnný. Můžeme však z řetězců vytvořit nový jejich spojením pomocí operátoru `+` nebo `*` (ano, násobení).

```
st1 = "ab" + "cd"      # abcd
st2 = "AB" + "CD"      # ABCD
st2 += st1              # ABCDabcd
st2 *= 2                # ABCDabcdABCDabcd
st2 = "*" * 10          # ***** – deset hvězdiček
```


Čtení znaků a vybírání podřetězců

Čtení znaků a vybírání podřetězců řetězce je stejné jako u seznamu, tj. pomocí *indexu* nebo *výřezu* (*slice*). Podřetězce ani znaky však *nelze na místě měnit*.

```
stl = "abcd"
stl[1]      # b
stl[-1]     # d
stl[4]      # chyba: string index out of range
stl[1:3]    # bc
stl[1:50]   # bcd – překročení rozsahu nezpůsobí chybu
stl[1:1]    #  – prázdný znak
```

Zvláštní znaky se zpětným lomítkem

Uvnitř zápisu řetězcového literálu lze zapisovat některé dvojice znaků (tvaru *zpětné lomítko* a znak), které se přímo interpretují nebo zobrazí znak za lomítkem.

```
print("abcd\nef") # \n nový řádek
abcd
ef

print("abcd\tef") # \t tabulátor
abcd ef

print("abcd\\ef") # \\ zpětné lomítko
abcd\ef

print("abcd\"ef") # \" uvozovka
abcd"ef

print('abcd\'ef') # \' apostrof
abcd'ef
```

Zpětným lomítkem v příkazu lze spojit části řetězce.

```
print("velmi dlouhý text, velmi dlouhý te\
xt, velmi dlouhý text, velmi dlouhý text, ")

velmi dlouhý text, velmi dlouhý text, velmi dlouhý text, velmi dlouhý text,
```

Hrubý řetězcový literál

Hrubý řetězcový literál (raw literal string) se zapisuje s předřazeným písmenem **r** nebo **R** a považuje zpětné lomítko za běžný znak, takže výše zmíněné zvláštní znaky se neinterpretují, ale zobrazují.

```
print(r"abcd\nef") # zobrazí se i oba znaky \n
abcd\nef
```

```
print(r"abcd\"ef") # zobrazí se i oba znaky \"
abcd\"ef
```

Funkce repr()

Funkce **repr()** použitá na řetězec (ve formě literálu nebo jména proměnné) zobrazí řetězec v reprezentačním tvaru uzavřený mezi apostrofy.

```
print(repr("abcd\nef")) # nový řádek se neprovede, ale zobrazí se \n
'abcd\nef'
```

```
print(repr("abcd\"ef")) # zobrazí se uvozovka uvnitř uvozovek (lomítko ne)
'abcd"ef'
```

Vyhledávání v řetězci

```
ret = """My tři králové jdeme k vám, štěstí, zdraví vinšujem vám.
Štěstí, zdraví, dlouhá léta, my jsme k vám přišli z daleka."""
```

Zjištění přítomnosti podřetězce

Operátor **in** zjistí, zda hledaný řetězec je v prohledávaném řetězci přítomen. Nepřítomnost se zjistí záporom **not in**.

```
print("tři" in ret) # True
print("tři" not in ret) # False
```

Hledání indexů všech výskytů jediného znaku

1. Hledání příkazem **for-in** s indexem.

```
index = 0
for znak in ret:
    if znak == "ř":
        print(index, end=" ")
    index += 1
```

4 102

2. Hledání příkazem `for-in` s funkcí `enumerate()`.

Funkce `enumerate` umožňuje očíslovat prvky řetězce.

`enumerate(iterable, start=0)`

Parametr *iterable* je obecně objekt schopný iterace čili opakování, tj. prohlížení prvek po prvku, tedy posloupnost (nejen textový řetězec). Parametr *start* umožňuje zahájit číslování od určeného indexu. Funkce použitá v cyklu `for-in` vrací postupně dvojice (*index*, *prvek*) počínaje od (*start*, *iterable[0]*).

```
ret = """My tři králové jdeme k vám, štěstí, zdraví vinšujem vám.  
Štěstí, zdraví, dlouhá léta, my jsme k vám přišli z daleka."""
```

```
for index, znak in enumerate(ret): # první musí být zapsán index  
    if znak == "ř":  
        print(index, end=" ")
```

4 102

3. Hledání příkazem `while`.

```
index = 0  
while index < len(ret):  
    if ret[index] == "ř":  
        print(index, end=" ")  
    index += 1
```

4 102

Hledání indexů podřetězce znak po znaku

```
i = 0  
while i < len(ret):  
    if ret[i: i + len('ři')] == 'ři': # slice  
        print(i, end=" ")  
    i += 1
```

4 102

Hledání indexů podřetězce metodou `find()`

```
ret = """My tři králové jdeme k vám, štěstí, zdraví vinšujem vám.  
Štěstí, zdraví, dlouhá léta, my jsme k vám přišli z daleka."""
```

```
str.find(string[, start=0[, end=len(string)]])
```

Metoda `find()` vrací *index* prvního znaku řetězce *string*, jestliže ho najde v řetězci *str*. Jinak vrátí `-1`. Rozsah hledání lze omezit zadáním počátečního indexu *start* nebo koncového indexu *end*.

Hledání prvního výskytu.

```
index = ret.find("vá")    # 23
```

Hledání všech výskytů.

```
index, start = 0, 0  
while index != -1:      # nerovná-li se -1  
    index = ret.find("vá", start)  
    if index != -1:      # zamezím tisku -1  
        print(index, end=" ") # 23 52 97  
        start = index + 1
```

Poznámka: Funkci se říká *metoda*, když se píše za jménem objektu s tečkou. Je to tím, že pochází z definice třídy (class), kde jsou funkce konstruovány speciálním způsobem. Objekt typu `str` je produktem třídy (class) `str`. O třídách a metodách se pojednává v další části kurzu.

Cvičení

Napište funkci `indexes(sub, string)`, která najde všechny výskyty podřetězce (nebo jediného znaku) *sub* v řetězci *string* a jeho indexy umístí do seznamu, který vrátí. Vyvolejte funkci se zvolenými hodnotami argumentů a vrácený seznam vytiskněte.

Porovnání řetězců

Dva řetězce se porovnávají *lexikograficky*, tedy podle velikosti stejnohlých znaků (na stejném indexu). Má-li jeden řetězec méně znaků a ty se shodují se stejnohlými znaky delšího řetězce, považuje se kratší řetězec za menší. Znaky se porovnávají podle velikosti čísel, která jsou pro ně stanovena v soustavě Unicode.

```
str1 = "abcde"
str2 = "abč"

if str1 < str2:
    print(str1, "<", str2)
elif str1 > str2:
    print(str1, ">", str2)
else:
    print(str1, "==", str2)
abcde < abč

"abcde" < "abč"
"abcde" > "abc"
"abcde" < "abczz"
"abcde" > "Abcde"
"abcde" == "abcde"
```

Toto porovnání však zvyklostem v národních jazycích nevyhovuje:

```
"zč" < "čz"
"ch" < "h"
```

Proto se zavádí prostředí zvané *locale* (místní). Ani to však v Pythonu nevyhovuje všem jazykům, například češtině. Existují knihovní moduly, v nichž se tento problém řeší, například zde:

<https://pypi.org/project/czech-sort/>
<https://pypi.org/project/PyICU/>

Toto téma se bude probírat v další části kurzu.

Proměna řetězce na seznam, n-tici nebo množinu

Použijí se k tomu standardní funkce `list()`, `tuple()`, `set()`.

```
print(list("ABCDE"))    # ['A', 'B', 'C', 'D', 'E']
print(tuple("ABCDE"))   # ('A', 'B', 'C', 'D', 'E')
print(set("ABCDEABCDE")) # {'C', 'B', 'E', 'A', 'D'}
```

Více o rozhodování

Podmínka jako logický výraz

Podmínka je vlastně otázka, zda je nějaké tvrzení je pravda nebo ne. V životě jsou tvrzení, která nelze jednoznačně vyhodnotit, například "vláda vládne dobře". Někdo tvrdí, že ano, jiný že ne, další nevím nebo tak napůl, apod. Takové otázky se dávají v sociologických průzkumech, ale v matematice, ani v počítačovém programu se nepřipouštějí. Tam musí být jasné, zda odpověď je ano či ne.

V jazyku Python se tvrzení nazývá podmínkový nebo *logický výraz*, který může mít jen hodnotu ano (**True**) nebo ne (**False**). Takový výraz představuje porovnání nějakých hodnot mezi sebou.

Například tvrzení "tři jsou víc než dva" je pravdivé. Logický výraz $3 > 2$ má tedy hodnotu **True**, zatímco opačné tvrzení "tři jsou méně než dva" nikoliv; logický výraz $3 < 2$ má tedy hodnotu **False**. Hodnoty **True** a **False** se nazývají *pravdivostní hodnoty*.

V Pythonu má ale hodnotu **False** také číslo nula, prázdný řetězec, prázdný seznam a jiné prázdné objekty, zatímco ostatní mají hodnotu **True**. Zkoušku lze provést funkcí `bool()`.

```
print( bool(0) )   # False
print( bool(1) )   # True
print( bool([]) )   # False
print( bool(['a']) ) # True
```

Podobně jako aritmetické výrazy obsahují *logické výrazy* operandy a operátory.

Operandy jsou objekty schopné porovnání, tedy čísla, řetězce, seznamy aj. Mohou vystupovat jednotlivě nebo ve výrazech, např. v aritmetickém výrazu nebo řetězcovém výrazu.

Operátory jsou jednak porovnávací čili *relační* a jednak *logické*.

Relační operátory

<code>==</code>	rovno (Pozor, dvě rovnítka! Jedno rovnítko by znamenalo operátor přiřazení)
<code>!=</code>	nerovno (vykřičník a rovnítko)
<code><</code>	menší
<code>></code>	větší
<code><=</code>	menší nebo rovno
<code>>=</code>	větší nebo rovno
<code>is</code>	rovno (jako <code>==</code>)
<code>is not</code>	nerovno (jako <code>!=</code>)

Logické operátory

<code>and</code>	a zároveň (logický součin)
<code>or</code>	nebo (logický součet)
<code>not</code>	ne (logický zápor, negace)

Logické výrazy

Pršelo *nebo* sněžilo. Logický součet.

x or y
 bud' platí x a neplatí y,
 nebo neplatí x a platí y,
 anebo platí oboje

Pršelo *a* svítilo slunce. Logický součin.

x and y
 platí x a zároveň platí y

Nepršelo. Logický zápor - negace.

not x
 neplatí x

Tvrzení

pršelo *nebo* sněžilo
pršelo *a* svítilo slunce

Zápor

nepršelo *a* nesněžilo (nepršelo, *ani* nesněžilo)
nepršelo *nebo* nesvítilo slunce

Pro zápor logického součtu a součinu platí De Morganova pravidla.

not (x or y) ↔ **(not x) and (not y)**
not (x and y) ↔ **(not x) or (not y)**

Složené relační operátory:

a <= b ↔ **a < b or a == b.**
a >= b ↔ **a > b or a == b.**
a != b ↔ **not a == b.**

Příklad:

3 >= 2 True
3 <= 2 False

Vnořený příkaz *if* je k vnějšímu ve vztahu **and**.

```
if x < y:                ↔      if x < y and y < z:
    if y < z:              print(x < z)
        print(x < z)
```

Relační operátory se *řetěží* ve vztahu **and**.

a < b < c ↔ **a < b and b < c**

Složitější logické výrazy

Operátor `and` váže těsněji než operátor `or`. Operátor `not` váže ještě těsněji. *Závorky* však vážou nejtěsněji, proto se pro určitost části logického výrazu uzavírají do závorek podobně jako u aritmetického výrazu. V tom případě se nejprve vyhodnocují obsahy závorek.

```
a, b = 1, 2
```

```
a == 1 and b == 2 or a == 2 and b == 3    # True
(a == 1 and b == 2) or (a == 2 and b == 3) # True
```

```
a == 1 or a == 2 and b == 2 or b == 3    # True
a == 1 or (a == 2 and b == 2) or b == 3   # True
```

```
not a == 1 or not b == 3    # True
not (a == 1 and b == 3)    # True
```

```
not a == 1 and not b == 3   # False
not (a == 1 or b == 3)     # False
```

Ukázky s operátorem `not` demonstrují pravidla pro zápor (De Morganova pravidla). Negace logického součinu dává logický součet negací. Negace logického součtu dává logický součin negací.

Aritmeticko-logický výraz

Hodnota aritmetického výrazu se také považuje za pravdivostní. *Nenulová* hodnota je `True`, *nula* je `False`.

```
a, b, c = 1, 2, 3
if a + b - c:
    print('pravda')
else:
    print('lež')
```

```
lež
```

Podmínkový výraz

Kromě podmínkových příkazů `if-else` má Python také *podmínkový výraz*:

```
hodnota-1 if podmínka else hodnota-2
```

Výraz bude mít hodnotu *hodnota-1*, jestliže *podmínka* je splněna, jinak bude mít hodnotu *hodnota-2*.

```
a, b = 1, 50
print(a if a > 4.5 else b)
```

```
50
```


Slovník (dictionary)

Slovník (anglicky *dictionary*) obsahuje dvojice objektů zvané klíč a hodnota (key, value). Literál se zapisuje ve *složených* závorkách (curly braces). Dvojice mají tvar **klíč: hodnota** a jsou mezi sebou odděleny čárkou. Čárka za poslední dvojicí se připouští a ignoruje se.

```
znaky = {"a": 97, "b": 98, "c": 99, "d": 100, "e": 101,}
```

Klíčem první položky je řetězec "a", hodnotou je číslo 97. Slovník trochu připomíná seznam. Ale zatímco seznam je indexován celými čísly, slovník je *indexován klíči*, přičemž klíčem může být každý neměnný objekt, například řetězec nebo číslo, ne však seznam, ani jiný slovník.

Prázdný slovník se zapisuje jako dvojice složených závorek {} nebo jako volání třídy **dict()**.

```
cenik = {"jablka": 50.5, "hrušky": 67.00, "švestky": 30.99,
        "broskve": 85.00,}
print(cenik)
{'jablka': 50.5, 'hrušky': 67.0, 'švestky': 30.99, 'broskve': 85.0}
```

Počet položek (dvojic) určí funkce `len()`.

```
len(cenik) # 4
```

Čtení hodnoty podle klíče

```
cena = cenik["broskve"]
cena = cenik.get("broskve") # metoda get() místo []
print(cena)                # 85.0 (když se klíč našel)

cena = cenik["jahody"]     # chyba: KeyError: 'jahody' (když se klíč nenašel)
```

Změna hodnoty podle klíče

```
cenik["broskve"] = 98.70
print(cenik["broskve"]) # 98.7 když se klíč najde, opraví se hodnota
```

Přidání nové položky na konec

```
cenik["jahody"] = 112.00 # když se klíč nenajde, přidá se celá položka
print(cenik)
{'jablka': 50.5, 'hrušky': 67.0, 'švestky': 30.99, 'broskve': 98.7, 'jahody': 112.0}
```

Odebrání položky podle klíče – funkce `del()`

```
del(cenik["jahody"]) # funkce del() - delete
print(cenik)
{'jablka': 50.5, 'hrušky': 67.0, 'švestky': 30.99, 'broskve': 98.7}
```

Poznámka: Funkce `del()` pro slovník *není* shodná s příkazem `del` pro seznam.

Zjišťování informací o klíčích, hodnotách a položkách

Metody `keys()`, `values()`, `items()` vracejí objekty typu `dict_keys`, `dict_values`, `dict_items` zvané *views* (pohledy). Pohledy nejsou přímé seznamy, jsou to jen odkazy na obsah slovníku. Z pohledů lze zjišťovat informace o stavu slovníku, aniž bychom je převáděli do seznamů. Pohledy lze ovšem také převést na seznamy.

```
cenik = {"jablka": 50.5, "hrušky": 67.00, "švestky": 30.99, "broskve": 85.00,}
```

```
print(cenik.keys()) # dict_keys(['jablka', 'hrušky', 'švestky', 'broskve'])
print(cenik.values()) # dict_values([50.5, 67.0, 30.99, 98.7])
print(cenik.items()) # dict_items([('jablka', 50.5), ('hrušky', 67.0),
('švestky', 30.99), ('broskve', 98.7)])
```

```
for key in keys: # výpis klíčů
    print(key)
```

```
jablka
hrušky
švestky
broskve
```

```
for value in cenik.values(): # výpis hodnot
    print(value)
```

```
50.5
67.0
30.99
98.7
```

```
for item in cenik.items(): # výpis položek jako dvojic - dvojčlenných n-tic
    print(item)
```

```
('jablka', 50.5)
('hrušky', 67.0)
('švestky', 30.99)
('broskve', 98.7)
```

```
for key, value in cenik.items(): # výpis klíčů a hodnot z rozbalených dvojic
    print(key, value)
```

```
jablka 50.5
hrušky 67.0
švestky 30.99
broskve 98.7
```

Převod složek slovníku do seznamu

```
cenik = {'jablka': 50.5, 'hrušky': 67.0, 'švestky': 30.99, 'broskve': 98.7}
```

```
s = list(cenik) # převod klíčů funkcí list() do seznamu
print(s)
['jablka', 'hrušky', 'švestky', 'broskve']
```

```
s = [] # převod hodnot ze slovníku do seznamu
for val in cenik:
    s.append(cenik[val])
print(s)
[50.5, 67.0, 30.99, 98.7]
```

```
s = [] # převod dvojic ze slovníku do dvojic v seznamu
for elem in cenik.items():
    s.append(elem)
print(s)
[('jablka', 50.5), ('hrušky', 67.0), ('švestky', 30.99), ('broskve', 98.7)]
```

Cvičení

Ze slovníku

```
znaky = {"a": 97, "b": 98, "c": 99, "d": 100, "e": 101,}
```

- a) vytiskněte zvlášť všechny klíče, hodnoty a položky (dvojice),
- b) sečtěte všechny hodnoty.

Číselné soustavy

Ve výpočetní technice jsou často používány tyto číselné soustavy:

- desítková (dekadická, decimální)
- dvojková (binární)
- šestnáctková (hexadecimální)
- osmičková (oktalová) – tu pro stručnost vynecháme

Dvojková (binární) soustava má 2 číslice:

0 1

Šestnáctková (hexadecimální) má 16 číslic:

0 1 2 3 4 5 6 7 8 9 a b c d e f

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 – odpovídající desítkové číslice

Bajt (byte) je nejmenší adresovací jednotka v soudobých počítačích a také nejmenší jednotka měření objemů dat. Bajt je reprezentován dvojkovým číslem o 8 číslicích, např. $11010110_{(2)}$. Toto dvojkové (binární) číslo lze vyjádřit v ostatních soustavách. Příklady na převody mezi soustavami ilustrujeme na číslech o velikosti bajtu.

Převod celého čísla z dvojkové do desítkové soustavy

Stačí vzít jednotlivé číslice zleva (nebo zprava), vynásobit je příslušnou mocninou dvojky a výsledky sečíst.

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
dec	128	64	32	16	8	4	2	1	mocniny 2
	x	x	x	x	x	x	x	x	
bin	1	1	0	1	0	1	1	0	
	↓	↓	↓	↓	↓	↓	↓	↓	
dec	128	64	0	16	0	4	2	0	$\Sigma = 214_{(10)}$

Převod celého čísla z dvojkové do šestnáctkové soustavy

Převod stejného čísla $11010110_{(2)}$ z dvojkové do šestnáctkové soustavy je podobný, jen je potřeba umět šestnáctková čísla sčítat – podle šestnáctkové součtové tabulky:

+	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	10
2	3	4	5	6	7	8	9	a	b	c	d	e	f	10	11
3	4	5	6	7	8	9	a	b	c	d	e	f	10	11	12
4	5	6	7	8	9	a	b	c	d	e	f	10	11	12	13
5	6	7	8	9	a	b	c	d	e	f	10	11	12	13	14
6	7	8	9	a	b	c	d	e	f	10	11	12	13	14	15
7	8	9	a	b	c	d	e	f	10	11	12	13	14	15	16
8	9	a	b	c	d	e	f	10	11	12	13	14	15	16	17
9	a	b	c	d	e	f	10	11	12	13	14	15	16	17	18
a	b	c	d	e	f	10	11	12	13	14	15	16	17	18	19
b	c	d	e	f	10	11	12	13	14	15	16	17	18	19	1a
c	d	e	f	10	11	12	13	14	15	16	17	18	19	1a	1b
d	e	f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c
e	f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d
f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
hex	80	40	20	10	8	4	2	1	mocniny 2
	x	x	x	x	x	x	x	x	
bin	1	1	0	1	0	1	1	0	
	↓	↓	↓	↓	↓	↓	↓	↓	
hex	80	40	0	10	0	4	2	0	$\Sigma = d6_{(16)}$

Při sčítání se postupuje stejně jako v desítkové soustavě, přenos jedničky o řád se provede, když součet číslic překročí nejvyšší číslici f. Např. $bca + de = ca8$:

$$\begin{array}{r}
 \text{b c a} \\
 + \text{ d e} \\
 \hline
 \text{1 8} \\
 \text{1 9} \\
 \text{b} \\
 \hline
 = \text{c a 8}
 \end{array}$$

Zjednodušený převod mezi dvojkovou a šestnáctkovou soustavou

Převod mezi dvojkovou a šestnáctkovou soustavou (tam i zpět) je ještě zjednodušen tím, že $16 = 2^4$. To znamená, že *čtveřice* dvojkových číslic tvoří jednu šestnáctkovou číslici:

0001 = 1	1010 = a
0010 = 2	1011 = b
0011 = 3	1100 = c
0100 = 4	1101 = d
0101 = 5	1110 = e
0110 = 6	1111 = f
0111 = 7	
1000 = 8	
1001 = 9	

Největší číslo v bajtu je $11111111_{(2)}$ nebo $ff_{(16)}$, čili $255_{(10)}$. V Pythonu se základ soustavy vyjadřuje předponou `0b` u binární, `0x` u šestnáctkové (hexadecimální):

$255 = 0b11111111 = 0xff$

Příklad převodu z desítkové do šestnáctkové soustavy

Převody z dvojkové a šestnáctkové soustavy do desítkové jsou celkem přímočaré. Opačné převody jsou trochu náročnější, vyžadují operace postupného dělení se zbytkem.

Převod čísla 572 do šestnáctkové soustavy postupným dělením a odečítáním zbytku.

$572 : 16 = 30$	$35 : 16 = 2$	$2 : 16 = 0$
<u>-480</u>	<u>-32</u>	zbytek 2 = 2
92 : 16 = 5	3 : 16 = 0	
<u>-80</u>	zbytek 3 = 3	
12 : 16 = 0		
zbytek 12 = c		

Výsledné šestnáctkové číslice seřadíme *obráceně*: **23c**. Je tedy $572_{(10)} = 23c_{(16)}$.

Příklad převodu z desítkové do dvojkové soustavy

Převod čísla 14 do dvojkové soustavy postupným dělením a odečítáním zbytku.

$$\begin{array}{rcll} 141 & \% & 2 & = 1 \\ \hline -1 & & & \\ 140 & : & 2 & = 70 \% 2 = 0 \\ & \hline & -0 & & \\ & 70 & : & 2 = 35 \% 2 = 1 \\ & & \hline & & -1 & \\ & & 34 & : 2 = 17 \% 2 = 1 \\ & & & \hline & & & -1 \\ & & & 16 : 2 = 8 \% 2 = 0 \\ & & & \hline & & & -0 \\ & & & 8 : 2 = 4 \% 2 = 0 \\ & & & \hline & & & -0 \\ & & & 4 : 2 = 2 \% 2 = 0 \\ & & & \hline & & & -0 \\ & & & 2 : 2 = 1 \% 2 = 1 \\ & & & \hline & & & -1 \\ & & & 0 \end{array}$$

Výsledné dvojkové číslice seřadíme *obráceně*: **10001101**. Je tedy $141_{(10)} = 10001101_{(2)}$.

Cvičení

Desítkové číslo `dec` (např. 13) převedeme na dvojkový tvar vlastním programem.

1. Vytvoříme prázdný seznam.
2. Zjistíme zbytek po dělení čísla `dec` dvěma. Zbytek přidáme do seznamu.
3. Od čísla `dec` odečteme zbytek.
4. Rozdíl dělíme dvěma a dosadíme do `dec`.
5. Opakujeme od bodu 2, dokud číslo `dec` je větší nebo rovno 1.

Ve výsledném seznamu jsou dvojkové číslice v obráceném pořadí. Použitím metody `reverse()` na seznam číslic se pořadí v seznamu obrátí (na místě) a jeho položky můžeme vytisknout cyklem v pořadí od nejvyšších řádů. Mělo by vyjít 1101.

Převodní funkce `bin()` a `hex()`

Tyto standardní funkce usnadňují převody *celých* čísel mezi soustavami.

Převod celého desítkového čísla do dvojkové a šestnáctkové soustavy.

```
print(bin(3287))          # 0b110011010111
print(hex(3287))          # 0xcd7
```

Převod celého dvojkového čísla do šestnáctkové a desítkové soustavy.

```
print(hex(0b11111111))    # 0xff
print(0b11111111)         # 255
```

Převod celého šestnáctkového čísla do dvojkové a desítkové soustavy.

```
print(bin(0xff))          # 0b11111111
print(0xff)               # 255
```


Úprava dat pro výstup

Standardní funkce pro úpravu výstupu

Funkce `print()`

Dosud jsme pro výstup dat používali funkci `print()`, která má poměrně bohaté možnosti, jak jsme už vyzkoušeli. Tato funkce je definována takto:

```
print(*objects, sep=' ', end='\n', file=sys.stdout)
```

***objects** opakované objekty (viz hvězdičku na začátku), které jsou schopny převodu do řetězce.
sep oddělovací řetězec (separátor), kterým se oddělí jednotlivé objekty; předvolená hodnota je mezera.
end koncový řetězec za tiskem; předvolená hodnota je `'\n'` (nový řádek).
file výstupní soubor, do nějž tisk směřuje; předvolená hodnota je `sys.stdout` (standardní výstup).

```
print(1,2,3, sep=' - ', end=' ***')
```

```
1 - 2 - 3 ***
```

Poznámka: Název `print` (tisk) nemá nic společného s tiskárnou, ani s papírem. Jmenuje se tak setrvačně z doby, kdy se výstup z počítače opravdu tiskl na papír v řádkové tiskárně nebo v psacím stroji.

Převod objektu na řetězec

str() převádí objekt do podoby čitelné pro uživatele.
repr() převádí objekt do podoby obsahující všechny informace (pro programátora).

```
str(2/3)
```

```
0.6666666666666666
```

```
str("aaa\nbbb")    # provede odřádkování
```

```
aaa  
bbb
```

```
repr("aaa\nbbb")    # zobrazí uvozovky a kód nového řádku \n
```

```
'aaa\nbbb'
```

Převody znaků a pořadových čísel Unicode

chr() převod pořadového čísla Unicode na znak

ord() převod znaku na pořadové číslo Unicode

Vytiskneme znaky dané posloupností desítkových čísel 101 až 106 společně s čísly v šestnáctkovém tvaru.

```
for i in range(101, 106):  
    print(chr(i), hex(ord(chr(i))), sep=" ", end=" | ")
```

```
e 0x65 | f 0x66 | g 0x67 | h 0x68 | i 0x69 |
```

Vytiskneme znaky z řetězce "efghi" společně s jejich čísly v desítkovém tvaru.

```
for i in "efghi":  
    print(i, ord(i), sep=" ", end=" | ")
```

```
e 101 | f 102 | g 103 | h 104 | i 105 |
```

Formátované řetězcové literály

Python má několik historicky vzniklých formátovacích způsobů. Následující příklad je uvádí v pořadí jejich uvedení do jazyka. V příkladu ovšem nejsou žádné zvláštní formátovací prvky. Je jen naznačena struktura jednotlivých způsobů.

```
ovoce = "Pomeranče"
cena = 25

print(ovoce, "mají cenu", cena, "korun za kilo.")
print("%s mají cenu %d korun za kilo." % (ovoce, cena))
print("{} mají cenu {} korun za kilo.".format(ovoce, cena))
print(f"{ovoce} mají cenu {cena} korun za kilo.")
```

Pomeranče mají cenu 25 korun za kilo.

- První metoda je prosté použití funkce `print()`.
- Druhá metoda je podobná funkci `printf()` jazyka C.
- Třetí metoda zavádí do formátového řetězce složené závorky `{ }` na pozicích pro vložení dat zadaných ve funkci `format()` v odpovídajícím pořadí.
- Čtvrtá, zatím nejnovější metoda, označená písmenem **f** nebo **F**, využívá složené závorky `{ }` přímo k umístění dat.

Zde budeme probírat tu poslední metodu zvanou "formatted string literals" – formátované řetězcové literály.

Tvar formátovaného literálu

Formátovaný literál je řetězec, který obsahuje výrazy uvnitř složených závorek `{ }`. Má přibližný tvar

```
f" text { formátovaný-výraz } text { formátovaný-výraz } ... "
```

kde *text* značí znaky, písmeno **f** může být i velké **F**. *formátovaný-výraz* ve složených závorkách je zpravidla číselný a může být doplněn formátovými značkami. Uvozovky mohou být dvojité, jednoduché i trojitě. Složené závorky s formátovaným výrazem se v dokumentaci nazývají "replacement fields" čili *náhradová pole*. Náhradové pole má tvar

```
{ výraz, výraz, ... [=] [konverze] [: formátová-specifikace] }
```

- *Výraz* může být jednoduché jméno, řetězec, volání funkce, aritmeticko-logický výraz aj.
- Rovnítko způsobí otisk výrazu na výstupu.
- *konverze* dat může být jeden ze symbolů `!r`, `!s`, `!a`, které odpovídají funkcím `repr()`, `str()`, `ascii()`.
- *Formátová specifikace* za dvojtečkou je řetězec modifikující tvar hodnoty výrazů popsany dále.

Tvar formátové specifikace

`[ [výplň] zarovnání ] [ znaménko ] [ # ] [ 0 ] [ šířka ] [ seskupení ] [ . přesnost ] [ typ ]`

- **výplň** znak pro vyplnění prostoru zbylého po zarovnání (vlevo nebo vpravo)
- **zarovnání** hodnoty v rámci šířky prostoru:
< vlevo, > vpravo, ^ doprostřed, = vkládá výplň před číslo se znaménkem
- **znaménko** + tiskne kladné i záporné, – tiskne jen záporné (standardně), mezera jako –,
- **#** alternativní forma pro konverzi
- **0** výplň nulami před číslem; stejné jako zarovnání s výplní znakem '0'
- **šířka** prostoru pro výstup - minimální počet znaků
- **seskupení** oddělovač tisíců: _ (podtržítka) nebo , (čárka) nebo n (jen v lokalizovaném režimu)
- **.** desetinná tečka
- **přesnost** počet desetinných míst u typů f a F
počet platných číslic zleva po zaokrouhlení u typů g, G, n
maximální počet znaků u typu s.
- **typ** písmeno označující způsob zobrazení hodnoty (čísla, znaku)

Typ zobrazení

Pro typ `str`

`s` string (řetězec), stejné jako prázdný údaj

Pro typ `int`

- `b` binární formát
- `c` character (znak)
- `d` decimal integer (celé číslo)
- `o` oktalový (osmičkový)
- `x` hexadecimální malá písmena
- `X` hexadecimální velká písmena
- `n` jako `d`, ale v lokalizovaném režimu

Pro typy `float` a `Decimal`

- `e` vědecký tvar, např. 1.000000e+03
- `E` vědecký tvar, např. 1.000000E+03
- `f` tvar s pevnou tečkou, např. 1000.000000
- `F` jako `f`, ale místo nan (not a number) tiskne NAN a místo inf (infinity) tiskne INF
- `g` general - obecný tvar, který se přizpůsobuje přirozenému zobrazení dané hodnoty
- `G` jako `g`, ale přepne do `E`, je-li číslo příliš velké
- `n` jako `g`, ale v lokalizovaném režimu
- `%` procenta
- `None` (chybí zadání typu) je podobné typu `g`

Příklady formátování

Výrazy *slice* a volání funkce v náhradových polích

```
s = [324.65, 111, 211.50, 99.456]
print(f'Největší číslo v podseznamu {s[1:3]} je {max(s[1:3])}.')
```

Největší číslo v podseznamu [111, 211.5] je 211.5.

N-tice a podmínkový výraz ve výrazu

```
ovoce = "Pomeranče"
cena = 25
print(f'{ovoce, cena, cena if cena<20 else 20}')

('Pomeranče', 25, 20)
```

Otisk výrazu s výsledkem

```
print(f'{cena if cena<20 else 20 = }')
```

cena if cena<20 else 20 = 20

Formátové specifikace

Výplň 0, zarovnání vpravo, šířka 10

```
print(f'{256.77:0{10}} Kč') # 0000256.77 Kč
```

Výplň 0, zarovnání vpravo, šířka 10

```
print(f'{256.77:0>10}') # 0000256.77
```

Výplň *, zarovnání vpravo, šířka 10, znaménko vždy

```
print(f'{256.77:*>+10}') # ***+256.77
```

Výplň 0, zarovnání vlevo, šířka 10

```
print(f'{-256.77:<010}') # -256.77000
```

Číslo typu float zaokrouhlené na 2 des. místa

```
print(f'{150.5556:.2f}') # 150.56
```

Cvičení

Vytiskneme tabulku začátečních – neviditelných znaků Unicode s čísly 0 až 32 v hexadecimální a dekadické reprezentaci s odřádkováním po násobku 5. Použijeme funkce `chr()`, `ord()`, `repr()` a stanovení šířky sloupců (6 a 3) ve formátové specifikaci. Měla by se vytisknout takováto tabulka:

'\x00'	0	'\x01'	1	'\x02'	2	'\x03'	3	'\x04'	4	'\x05'	5
'\x06'	6	'\x07'	7	'\x08'	8	'\t'	9	'\n'	10		
'\x0b'	11	'\x0c'	12	'\r'	13	'\x0e'	14	'\x0f'	15		
'\x10'	16	'\x11'	17	'\x12'	18	'\x13'	19	'\x14'	20		
'\x15'	21	'\x16'	22	'\x17'	23	'\x18'	24	'\x19'	25		
'\x1a'	26	'\x1b'	27	'\x1c'	28	'\x1d'	29	'\x1e'	30		
'\x1f'	31	' '	32								

Soubory a adresáře

V počítačích jsou data uložena různě. Například jako literály v programech nebo odvozená data v operační paměti.

Data, která se nevejdou do operační paměti, nebo data, která tam nechceme uchovávat, se ukládají do tzv. datového úložiště (data repository) ve vnější paměti, což jsou lokální disky, vzdálené disky, komunikační linka apod. Objem dat se měří v bajtech (označených jako B). Používají se dvojí násobky počtu bajtů. Buď v desítkové mocnině $10^3 = 1000$ nebo dvojkové mocnině $2^{10} = 1024$.

Násobky 1000

KB = 1,000B,

MB = 1,000,000B

GB = 1,000,000,000B

TB = 1,000,000,000,000B

...

Násobky 1024

KiB = 1,024B

MiB = 1,048,576B

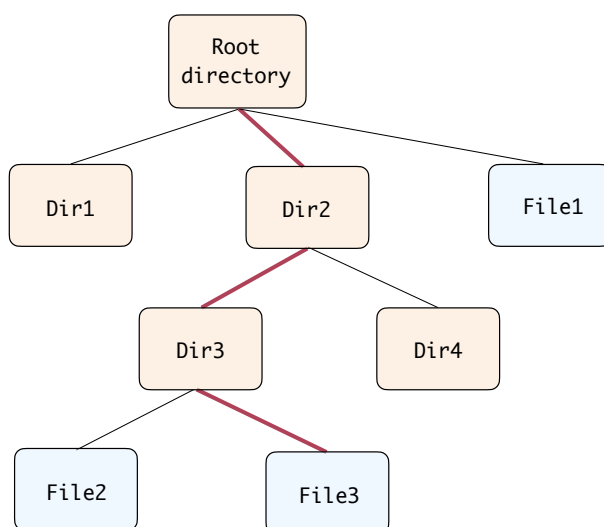
GiB = 1,073,741,824B

TiB = 1,099,511,627,776B

Předpokládáme, že vnější paměti jsou lokální disky. Data do nich uložená se člení na části zvané *soubory* nebo *dokumenty*.

Přístup k souborům je usnadněn jejich umístěním do hierarchické struktury *adresářů* zvaných také *složky* podle papírových složek s dokumenty. Adresáře můžeme považovat za rozcestníky, ukazatele nebo směrníky ukazující cestu, ale také za nádoby (kontejnery) obsahující další objekty. Anglické názvy jsou

- *directory* nebo *folder* pro *adresář*,
- *file* nebo *document* pro *soubor*.



Cesta (path)

K určení souboru nebo i adresáře v hierarchii slouží *cesta* (path). Plná, absolutní cesta se vyjádří jako posloupnost jmen adresářů počínaje nejvyšším, tzv. *kořenovým* (root) adresářem. Na obrázku je absolutní cesta k souboru File3 vyznačena: Root directory – Dir2 – Dir3 – File3.

V programech se používá domluvený zápis cesty, který se liší podle operačního systému.

Ve většině systémů typu UNIX se kořenový adresář označuje lomítkem `/` a ostatní adresáře a soubory jsou odděleny také lomítkem. Cesta od kořenového adresáře k souboru File3 se zapíše takto:

```
/Dir2/Dir3/File3
```

V systému Windows je kořenových adresářů více. Označují se písmenem (A, B, C, . . .) s dvojtečkou a zpětným lomítkem, nejčastěji `C:\`. Ostatní adresáře a soubory jsou odděleny zpětným lomítkem `\`. Cesta od kořenového adresáře k souboru File3 se zapíše takto:

```
C:\Dir2\Dir3\File3
```

Python však dovoluje i pro systém Windows použít unixovský způsob, který si interně přizpůsobí.

Jména adresářů a souborů mohou být psána s malými i velkými písmeny, ale při výpočtu se na velikost písmen nebere ohled. Jméno Dir1 může být i DIR1 nebo dir1 apod.

Absolutní a relativní cesta

Cesta k adresáři nebo souboru se nazývá *absolutní* nebo *úplná*, jestliže začíná kořenovým adresářem. Jinak se nazývá *relativní*. Programy potřebují k zadání souboru někdy absolutní cestu, někdy relativní cestu. Kterou část cesty zvolíme za relativní, záleží na konkrétní situaci v programu.

Například jako cestu k souboru File3 můžeme zvolit relativní cestu

Dir2/Dir3/File3	ke kořenovému adresáři /
Dir3/File3	k adresáři /Dir2/
File3	k adresáři /Dir2/Dir3/

Absolutní cesta je složena z absolutní cesty k nadřazenému adresáři a z připojené relativní cesty.

Běžný adresář (current directory)

Běžný nebo *současný* adresář je název pro adresář, který určitá aplikace (program) nebo operační systém považuje za nejčastěji používaný pro začátek absolutní cesty. Je-li běžný adresář určen, nemusíme ani znát jeho absolutní cestu a můžeme pracovat s *cestami relativními* k běžnému adresáři.

K práci s adresáři a cestami v Pythonu se používá systémový program (modul) nazvaný **os**. Ten je třeba zařadit příkazem **import os**.

Běžný adresář se zjišťuje funkcí **getcwd()** – get current working directory.

```
import os
curdir = os.getcwd() # zjištění běžného adresáře
print(curdir)        # C:\Users\vizupka\PycharmProjects\zaklady – Windows
                     # /Users/vizupka/PycharmProjects/zaklady – macOS
                     # /home/vizupka/PycharmProjects/zaklady – linux
```

Poznámka: Objekt (zde funkci **getcwd**) z importovaného modulu (zde **os**) vyvoláme s předřazeným jménem modulu a tečkou (**os.**). O modulech se pojednává v další části kurzu.

Vytvoření a smazání adresáře

```
import os
cesta = "DirA" # relativní cesta k novému adresáři
```

Vytvoříme adresář **DirA** v běžném adresáři funkcí **mkdir()** – *make directory*.

```
os.mkdir(cesta) # chyba FileExistsError, když už existuje
if os.path.exists(cesta):
    print(f"Adresář {cesta} vytvořen.")
```

Smažeme adresář **DirA** použitím funkce **rmdir()** – *remove directory*. Potom testujeme, zda adresář opravdu neexistuje, a to pomocí funkce **exists()**. Zde je **os.path** podmodul modulu **os**.

```
os.rmdir(cesta)
if not os.path.exists(cesta):
    print(f"Adresář {cesta} odstraněn.")
```

Poznámka: Objekty **getcwd**, **mkdir**, **rmdir** a **exists** jsou *funkce*. I když jsou volány s tečkou, nejsou to metody, protože **os** není třída, ale modul.

Vytvoření a otevření souboru

Nový soubor se *vytváří* (otevírá) standardní funkcí **open** (). Do něj pak můžeme zapisovat data a číst je z něj. Existující soubor se musí před zahájením zápisu dat nebo čtení dat také *otevřít* funkcí **open** ().

Poznámka: Funkce **open** () je standardní, vestavěná funkce. K práci se souborem není potřeba importovat modul **os**. (V modulu **os** existuje jiná funkce **open** (), kterou zde nepoužíváme.)

```
file_object = open(file, mode='r', buffering=-1, encoding=None,
                    errors=None, newline=None, closefd=True, opener=None)
```

Funkce **open** vrací objekt nazývaný soubor (file). V případě neúspěchu vyvolá chybu **OSError**. Z parametrů uvedeme jen některé.

file cesta k souboru zapsaná jako znakový řetězec

mode způsob otevření souboru

'r' *read* – pouze pro čtení (předvolený), stejné jako 'rt'
'w' *write* – pouze pro zápis, předem smaže obsah, stejné jako 'wt'
'x' *exclusive* – vytvoří nový soubor; existuje-li soubor, vyvolá chybu
'a' *append* – pouze pro přidávání na konec existujícího souboru
'b' *binary* – považuje obsah souboru za binární (složený z bajtů)
't' *text* – textový režim (předvolený)
'+' *update* – čtení i zápis

Jednotlivé znaky lze kombinovat: 'r+' čtení i zápis, 'w+' čtení i zápis (a smaže obsah), 'rb' čtení v binárním režimu, 'wb' zápis v binárním režimu apod.

encoding znakový řetězec označující kódování znaků v *textovém* souboru

Bez zadání tohoto argumentu se bere kódování znaků zavedené v operačním systému. Může to být

encoding='utf-8' (macOS)
encoding='cp1250' (Windows)
encoding='iso-8859-2' (unixové systémy).

pro střední a východní Evropu.

Kódování znaků v Pythonu je popsáno v článku [Codec registry and base classes](#). Česky si o kódování znaků můžeme přečíst [zde](#).

K další práci se používají *metody* **write** (zápis dat), **read** (čtení dat), **close** (uzavření).

Zápis dat do souboru funkcí `print()`

Textový soubor je znakový řetězec umístěný v datovém souboru. Do souboru můžeme texty snadno zapisovat standardní funkcí `print()`. Soubor otevřeme pro zápis, zapíšeme do něj od začátku text v kódu `utf-8` a soubor uzavřeme metodou `close()`.

Jméno `Text.txt` určuje *relativní cestu* vzhledem k běžnému adresáři.

```
rel_cesta = "Text.txt"
out = open(rel_cesta, mode="w", encoding="utf-8")
print("Nový text zapsaný do souboru od začátku.", file=out)
out.close() # uzavřu soubor
```

Vytvoření souboru a zápis textu metodou `write()`

Metoda `write()` zapíše řetězcový argument do souboru na následující pozici.

```
out = open("básnička.txt", "w")
out.write('''
Byl jeden domeček,
v tom domečku stoleček, ''')
out.write('''
na stolečku mistička,
v té mističce rybička. ''')
out.close()
```

Test existence a výmaz souboru

```
import os
rel_cesta = "Text.txt"
if os.path.exists(rel_cesta):
    print(f"Odstraňuji soubor {rel_cesta}.")
    os.remove(rel_cesta)
print(os.path.exists(rel_cesta)) # False
```

Čtení textového souboru po řádcích

V cyklu `while` můžeme číst jednotlivé řádky metodou **`readline()`**, dokud nenarazíme na *konec* signalizovaný *prázdným znakem* `" "`.

```
relpath = "básnička.txt"
infile = open(relpath)      # otevřu soubor pro čtení
line = infile.readline()    # čtu první řádek
while line:                 # není-li řádek prázdný, provedu blok
    print(line, end="")      # vytisknu přečtený řádek
    line = infile.readline() # čtu další řádek
infile.close()              # uzavřu soubor
```

Při čtení cyklem `for-in`, ani nemusíme testovat, zda není konec souboru. Soubor se zde považuje za posloupnost řádků schopnou iterace. Pro příklad použijeme *absolutní cestu* k souboru.

```
infile = open(
    "/Users/vzupka/PycharmProjects/zaklady/básnička.txt", "r")
for line in infile:
    print(line, end="")
infile.close()
```

Čtení textového souboru po jednom znaku

Vypíšeme obsah souboru *básnička.txt* pomocí metody **`read(1)`**, který čte další znak v pořadí.

```
f = open("básnička.txt")
znak = f.read(1)
while znak: # neprázdný znak?
    print(znak, sep=' ', end=' ')
    znak = f.read(1)
f.close()
```

Cvičení

- Vytvořte v běžném adresáři nový adresář "Nový adresář".
- V něm vytvořte nový textový soubor, do něhož zapíšete text "Nový soubor v novém adresáři" v kódu `utf-8`.
- Vypište obsah souboru.
- Odstraňte soubor z nového adresáře.
- Odstraňte nový adresář z běžného adresáře.

Chyby a výjimky

V programu mohou nastat chyby nebo výjimečné události, při nichž program může havarovat. Jde například o chybný typ dat nebo čtení z uzavřeného souboru aj. Máme-li důvodné podezření, že by v nějakém úseku programu mohlo dojít k chybě, snažíme se tomu vhodným způsobem vyhnout.

Zpracování souboru příkazem `with-as`

Proti chybě, která by nastala, kdybychom zapomněli uzavřít soubor a chtěli jej otevřít po druhé, se můžeme pojistit příkazem `with-as`. Ten otevře soubor, kontroluje výpočet v bloku, a na konci *zajistí uzavření souboru*.

```
with open("básnička.txt", encoding="utf-8", mode="r") as infile:
    pass # prázdný příkaz
if infile.closed:
    print("Soubor je uzavřen.")
```

Soubor je uzavřen.

Testování typu dat

Následující program upravuje jedním způsobem náš první příklad výpočtu maxima. Ke kontrole dat použijeme funkci `type(objekt)`, která vrací typ objektu. Operátor `is` pak zjistí, zda jde o požadovaný typ.

Abychom se vyhnuli chybě hned v první položce, použijeme funkci `float("-inf")` k dosazení nejmenšího záporného čísla (`inf` znamená infinity – nekonečno) do proměnné `maximum`.

Program zachytí položky s chybným typem, oznámí je a vynechá ze zpracování.

```
vstup = ["23", 32, 42, "20", 19, 6, "42"]
if len(vstup) == 0:
    print("Prázdný vstup")
    exit(1)
maximum = float("-inf") # nejmenší možné číslo
for n in range(len(vstup)):
    if type(vstup[n]) is int or type(vstup[n]) is float:
        if vstup[n] > maximum:
            maximum = vstup[n]
    else:
        print(f"Došlo k chybě v {n + 1}. položce \"{vstup[n]}\".")
print("maximum =", maximum)
```

```
Došlo k chybě v 1. položce "23".
Došlo k chybě v 4. položce "20".
Došlo k chybě v 7. položce "42".
maximum = 42
```

Testování chyb příkazem `try`

Python rozeznává řadu různých druhů *chyb* (errors) pod společným názvem *výjimky* (exceptions), např. `TypeError`, `FileNotFoundError`, `ValueError`, aj. Při výpočtu lze chyby zachycovat a ošetřovat příkazem `try`. Ten k sobě potřebuje ještě doprovodné příkazy – `except` nebo `finally`:

```
try:
    blok podezřelých příkazů
except výraz:      chytá konkrétní výjimky
    blok příkazů
...
except:           chytá zbývající výjimku
    blok příkazů
finally:         provede se vždy
    blok příkazů
```

kde *výraz* může být několik názvů výjimek v závorce, např. `(TypeError, ValueError)` nebo název výjimky se slovem `as` a proměnnou, např. `Exception as chyba`. Alespoň jeden z bloků `except` nebo blok `finally` musí být zapsán.

Příklad 1

Následující program upravuje náš příklad výpočtu maxima čísel jiným způsobem. Do bloku `try` umístíme podezřelé příkazy, do bloku `except` příkazy reagující na případnou chybu, aniž bychom předem stanovili, jaký druh chyby chceme zachytit.

```
vstup = ["-23", -32, -42, "-20", -19, -6, "-42"]
if len(vstup) == 0:
    print("Prázdný vstup")
    exit(1)
maximum = float("-inf")      # nejnižší možné číslo
for n in range(len(vstup)):
    try:
        if vstup[n] > maximum:
            maximum = vstup[n]
    except:
        print(f"Došlo k chybě v {n + 1}. položce \"{vstup[n]}\".")
print("maximum =", maximum)
```

```
Došlo k chybě v 1. položce "-23".
Došlo k chybě v 4. položce "-20".
Došlo k chybě v 7. položce "-42".
maximum = -6
```

Příklad 2

Použití výrazu s proměnnou v bloku **except**. Proměnná obsahuje typ zachycené výjimky a text zprávy. Název **Exception** je společný (generický) název pro řadu výjimek – chyb.

```
try:
    int('řetězec')    # převod řetězce na celé číslo
except Exception as err:
    print(type(err))  # typ výjimky
    print(err)        # text zprávy
```

```
<class 'ValueError'>
invalid literal for int() with base 10: 'řetězec'
```

Příklad 3

Použití bloku **finally**. Program potlačí a oznámí chybu při otevírání nebo zpracování souboru a nakonec uzavře soubor.

```
try:
    input = open('básnička.txt')
    input.write('a')    # zápis do čtecího souboru!
except Exception as err:
    print('Chyba:', err)
finally:
    input.close()       # po chybě uzavřu soubor
```

```
Chyba: not writable
```

Příklad 4

Standardní funkce **max()** funguje u posloupností s prvky jednotného typu, které umožňují porovnání na velikost, např. u řetězců.

```
ntice = ("23", "32", "42", "20", "19", "6", "42")
maximum = max(ntice)
print("maximum =", repr(maximum))    # maximum = '6'
```

Sama funkce **max()** ale nijak neusnadní zjistit místo (index) v seznamu, kde došlo k chybě.

```
seznam = ["23", 32, 42, "20", 19, 6, "42"]
try:
    maximum = max(seznam)
except Exception as err:
    print('Chyba:', err)
```

```
Chyba: '>' not supported between instances of 'int' and 'str'
```

Vyvolání výjimky

Programátor může také vyvolat výjimku, jestliže to může usnadnit použití jeho programu, zpravidla funkce. Výjimku lze vyvolat jedním ze dvou příkazů: **assert**, **raise**. Užitečná je zvláště kontrola správnosti argumentů ve volání funkce, zejména zda mají správný typ, délku nebo hodnotu. Potřebná je proto, že tyto podmínky nelze v Pythonu kontrolovat při kompilaci programu jako v jiných jazycích.

Kupříkladu následující funkce `kontrola()`, která pracuje se seznamem, vyžaduje pro kontrolu zadání jeho délky v parametru. Když kontrola vyjde, vše proběhne bez ohlášení. Nevyjde-li kontrola, funkce vyvolá výjimku a ohlásí příčinu tiskem.

Příkaz `assert`

Příkaz `assert` (tvrzení) požaduje, aby nějaký výraz byl pravdivý, a vyvolá výjimku `AssertionError`, když není pravdivý. Příkazů `assert` může být ve funkci zařazeno více, podle důkladnosti kontrol. Při výpočtu se uplatní ten, který nejdříve nevyjde a pak výpočet skončí.

```
def kontrola(seznam, delka):  
    assert type(seznam) == list, f"seznam: {seznam} není typu list!"  
    assert type(delka) == int, f"delka: {delka} není typu int!"  
    assert len(seznam) == delka, f"delka: má být {len(seznam)}, ale je {delka}!"
```

```
try:  
    kontrola((1, 1, 1, 1, 1), 4)  
except AssertionError as err:  
    print("AssertionError!", err)
```

AssertionError! seznam: (1, 1, 1, 1, 1) není typu list!

```
try:  
    kontrola([1,1,1,1,1], 4)  
except AssertionError as err:  
    print("AssertionError!", err)
```

AssertionError! Délka má být 5, ale je 4!

```
try:  
    kontrola([1, 1, 1, 1, 1], 10/2)  
except AssertionError as err:  
    print("AssertionError!", err)
```

AssertionError! delka: 5.0 není typu int!

Příkaz `raise`

Příkaz `raise` můžeme použít k vyvolání výjimky, třeba když není splněna nějaká podmínka. V příkladu vyvolá výjimku `Exception`. V příkladu je ke kontrole parametrů použita jedna složitá podmínka. Zatímco příkaz `assert` můžeme volat vícekrát, příkaz `raise` můžeme vyvolat jen jednou, protože po něm funkce končí.

```
def kontrola(seznam, delka):
    if type(seznam) != list or type(delka) != int or len(seznam) != delka:
        raise Exception(
            f"""
            seznam:{seznam} není typu list
            nebo delka: {delka} není typu int
            nebo delka není {len(seznam)}!"""
```

```
try:
    kontrola(seznam=(1, 1, 1, 1, 1), delka=4.0)
except Exception as err:
    print("Chyba v parametrech!", err)
```

```
Chyba v parametrech.
    seznam:(1, 1, 1, 1, 1) není typu list
    nebo delka: 4.0 není typu int
    nebo delka není 5!
```

Vstup dat z klávesnice – funkce `input()`

Ke vstupu dat z klávesnice slouží standardní funkce `input()`, která má tento tvar:

`input([návod])`

Funkce vytiskne *nepovinný návod* na konzolu (standardní výstup) a *čeká* na vstup z klávesnice. Když uživatel napíše nějaké údaje a stiskne klávesu *Enter*, vrátí funkce tyto údaje převedené na *textový řetězec*. Je-li vstup prázdný (když uživatel nenapíše žádné údaje a stiskne *Enter*), vrátí funkce *prázdný řetězec* jako příznak konce vstupu.

První příklad jen otiskne data ze vstupu.

Příklad zadaný z editoru

```
cislo = input("zadejte číslo: ")  
print(cislo)
```

Spustíme program a na konzole se objeví návod.

zadejte číslo:

Program čeká na vstup. Napíšeme k němu číslo 123 a stiskneme Enter. Na konzole se ukáže výsledek a program skončí.

```
zadejte číslo: 123  
123
```

Process finished with exit code 0

Příklad zadaný z příkazového řádku

```
>>> cislo = input("zadejte číslo: ") # příkaz zapsaný z klávesnice  
zadejte číslo: 123  
>>> cislo  
'123'  
>>>
```

Kontrola vstupu (zelená linka uvádí obraz konzoly)

Nesprávný typ vstupního údaje způsobí chybu.

```
cislo = input("zadejte číslo: ")
c = cislo + 1
print(cislo)
```

zadejte číslo: a

...

TypeError: can only concatenate str (not "int") to str

Vstupní data můžeme testovat na výjimku `TypeError`.

```
cislo = input("zadejte číslo: ")
try:
    c = cislo + 1
except TypeError as err:
    print(f"Chybný typ vstupních dat {cislo}.")
```

zadejte číslo: A123

Chybný typ vstupních dat A123.

Po chybném vstupu opakovaně vyžadujeme nový vstup. Po stisku Enter program končí.

```
cislo = input("Zadávejte sčítaná čísla:\n")
suma = 0
while True:
    try:
        c = int(cislo)
        suma += c
        cislo = input()
    except Exception:
        if cislo == '':
            break
        cislo = input(f"Chybně zadané číslo {cislo}.
Zadejte znovu správné číslo nebo ENTER pro konec:\n")
print("suma =", suma)
```

Zadávejte sčítaná čísla:

1

2

399

-399

a

Chybně zadané číslo a.

Zadejte znovu správné číslo nebo ENTER pro konec:

7

suma = 10

Výpočet maxima z čísel zadaných z klávesnice

```
maximum = float('-inf') # nejmenší možné číslo
index = 0
cislo = input(f""""Zadávejte jednotlivá čísla nebo více čísel oddělených čárkou.
(Vstup ukončete dalším stiskem klávesy Enter):\n""")
if not cislo:
    print("Prázdný vstup")
    exit(2)
while cislo:
    ssplit = cislo.split(",") # rozdělí řádek podle čárek na seznam
    for n in range(len(ssplit)): # převod prvků seznamu na čísla
        try:
            if ssplit[n] != "\n": # vynecháváme znak nový řádek
                fl = float(ssplit[n]) # převod řetězce na číslo
                if fl > maximum: # výpočet maxima
                    maximum = fl
        except Exception as err:
            print(f"Chyba v {index + 1}. "
                  f"řádku {n + 1}. prvku {repr(ssplit[n])}: {err}")
    cislo = input() # čte další znak ze vstupu
    index += 1
print("\nmaximum =", maximum)
```

Zadávejte jednotlivá čísla nebo více čísel oddělených čárkou.

(Vstup ukončete dalším stiskem klávesy Enter):

45

52

a

Chyba v 3. řádku 1. prvku 'a': could not convert string to float: 'a'

666

maximum = 666.0

Zadávejte jednotlivá čísla nebo více čísel oddělených čárkou.

(Vstup ukončete dalším stiskem klávesy Enter):

45, 52 , a , 666

Chyba v 1. řádku 3. prvku ' a ': could not convert string to float: ' a '

maximum = 666.0